TRƯỜNG ĐẠI HỌC CÔNG THƯƠNG TP. HỒ CHÍ MINH

KHOA CÔNG NGHỆ THÔNG TIN

TIỂU LUẬN BIG DATA

ĐỀ TÀI: Phân tích xu hướng âm nhạc trên Spotify (2017 – 2023)

Giảng viên hướng dẫn: Huỳnh Thái Học

Sinh viên thực hiện: Nhóm 9

1. Nguyễn Hà Trọng Phúc – 2001230686
2. Nguyễn Vũ Ngọc Nam – 2001230545
3. Lê Đông Triều – 2001230984

TP. Hồ Chí Minh, tháng 10, 2026

MỤC LỤC

LỜI MỞ ĐẦU 7
1. Lý do chọn đề tài 7
2. Mục tiêu của đề tài 7
3. Phạm vi nghiên cứu 8
4. Phương pháp nghiên cứu 8
5. Bố cục của tiểu luận 8
CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI VÀ DỮ LIỆU 9
1.1. Bối cảnh và tính cấp thiết của đề tài 9
1.2. Bộ dữ liệu sử dụng 9
1.2.1. Nguồn và quy mô dữ liệu 9
1.2.2. Cấu trúc dữ liệu 9
1.2.3. Đặc tả chi tiết 17 trường dữ liệu của bộ dữ liệu Spotify Top 200 10
1.3. Sản phẩm đầu ra của đề tài 11
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG 12
2.1. Big Data và hệ sinh thái Hadoop – Spark 12
2.1.1. Khái niệm Big Data 12
2.1.2. Hệ thống tệp phân tán HDFS 13
2.1.3. Apache Spark, PySpark và Spark SQL 13
2.2. Học máy với Spark MLlib 13
2.2.1. Phân tích RFM 13
2.2.2. Thuật toán K-Means 14
2.3. Apache Kafka và xử lý luồng dữ liệu 14
2.4. Apache Airflow và tự động hóa quy trình 14
2.5. Định dạng lưu trữ Parquet 14

CHƯƠNG 3: THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG 16
3.1. Kiến trúc tổng thể 16
3.2. Bước 1 – 3: Hạ tầng HDFS và PySpark 17
3.2.1. Tổ chức kho lưu trữ 17
3.2.2. Tải dữ liệu và cấp phát tài nguyên 17
3.2.3. Cơ chế dự phòng 17
3.2.4. Bảng thông số hạ tầng và nhật ký lệnh HDFS CLI thực tế 18
3.3. Bước 4 – 5: Tiền xử lý và Spark SQL 19
3.3.1. Các thao tác tiền xử lý 19
3.3.2. Khởi tạo SparkSession 20
3.3.3. Truy vấn chỉ số KPI bằng Spark SQL 21
3.3.4. Mã truy vấn phân tích dữ liệu Spark SQL nâng cao và bảng kết quả 21
3.4. Bước 6: Phân tích RFM và phân cụm K-Means 22
3.4.1. Phân khúc RFM âm nhạc 22
3.4.2. Phân cụm K-Means theo phong cách 23
3.4.3. Pipeline huấn luyện học máy PySpark MLlib và đánh giá Silhouette 23
3.5. Bước 7: Lưu trữ kết quả 24
3.6. Bước 8 – 9: Xử lý liên tục và tự động hóa 24
3.6.1. Xử lý thời gian thực với Kafka 24
3.6.2. Tự động hóa với Airflow 25
3.6.3. Mã nguồn mô phỏng Kafka Streaming Producer và Airflow DAG ... 26
CHƯƠNG 4: KẾT QUẢ VÀ ĐÁNH GIÁ 29
4.1. Sản phẩm thực tế: Web Dashboard 29
4.1.1. Giao diện và chức năng 29
4.1.2. Các chỉ số tổng quan 31

4.2. Các phát hiện từ dữ liệu 32
4.2.1. Biến động đặc trưng âm thanh giai đoạn 2017 – 2023 32
4.2.2. Ca khúc ngày càng ngắn lại 33
4.2.3. Bảng xếp hạng Top nghệ sĩ và độ phổ biến âm nhạc 33
4.2.4. Trực quan hóa không gian phân cụm học máy K-Means (PCA 2D) . 34
4.2.5. Giám sát luồng thông điệp phát nhạc thời gian thực qua Kafka 35
4.3. Đánh giá hệ thống 36
4.3.1. Ưu điểm..... 36
4.3.2. Hạn chế..... 37
4.3.3. Đánh giá tổng thể 37
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 39
5.1. Kết luận 39
5.2. Hạn chế của hệ thống 39
5.3. Hướng phát triển 39
TỔNG HỢP MINH CHỨNG VÀ KẾT QUẢ TRIỂN KHAI 40
TÀI LIỆU THAM KHẢO 42
PHỤ LỤC A: NHẬT KÝ TIẾN ĐỘ THỰC HIỆN ĐỒ ÁN (MILESTONES) 43
PHỤ LỤC B: CÁC LỖI KỸ THUẬT GẶP PHẢI VÀ CÁCH KHẮC PHỤC 43
PHỤ LỤC C: LỊCH SỬ COMMIT MÃ NGUỒN (GIT COMMIT HISTORY) 44
PHỤ LỤC D: BẢNG PHÂN CÔNG CÔNG VIỆC VÀ TỶ LỆ ĐÓNG GÓP 45
PHỤ LỤC E: CAM KẾT BẢO MẬT VÀ LIÊM CHÍNH HỌC THUẬT 45
PHỤ LỤC F: HƯỚNG DẪN CÀI ĐẶT VÀ TÁI LẬP HỆ THỐNG (README & REPRODUCIBILITY GUIDE) 47

DANH MỤC HÌNH

Hình 3.1: Kiến trúc data pipeline 9 bước 16

Hình 3.2: Tệp lệnh chạy công việc PySpark RFM và K-Means 18

Hình 3.3: Hàm khởi tạo SparkSession 20

Hình 3.4: Định nghĩa default_args và DAG trong Airflow 25

Hình 4.1: Giao diện Dashboard với vùng kéo thả tệp và các thẻ chỉ số tổng quan 30

Hình 4.1b: Giao diện tổng quan luồng dữ liệu và các chỉ số KPI trên Streamlit. 31

Hình 4.2: Biến động đặc trưng âm thanh giai đoạn 2017 – 2023 32

Hình 4.3: Xu hướng thời lượng trung bình của bài hát qua các năm 33

Hình 4.4: Bảng xếp hạng Top nghệ sĩ và phân tích độ phổ biến âm nhạc 34

Hình 4.5: Trực quan hóa không gian phân cụm bài hát K-Means (PCA 2D) 35

Hình 4.6: Giám sát luồng thông điệp phát nhạc thời gian thực qua Kafka 36

DANH MỤC BẢNG

Bảng 1.1: Các nhóm thuộc tính của bộ dữ liệu 10

Bảng 1.2: Cấu trúc Schema 17 trường dữ liệu của bộ dữ liệu Spotify Top 200 .. 10

Bảng 3.1: Các bước trong pipeline và công nghệ tương ứng..... 17

Bảng 3.2b: Thông số cấu hình hệ sinh thái Big Data của đề tài 18

Bảng 3.2: Các tham số cấu hình SparkSession 21

Bảng 3.3b: Kết quả thực thi các truy vấn Spark SQL tổng hợp theo từng năm... 22

Bảng 3.3: Ba tiêu chí RFM áp dụng cho bài hát 22

Bảng 3.4: Bốn nhóm phong cách do K-Means phân loại 23

Bảng 3.5: Các tham số chính của DAG 25

Bảng 4.1: Các chỉ số tổng quan hiển thị trên Dashboard..... 31

Bảng 4.2: Giá trị xấp xỉ của các đặc trưng âm học (%) 32

Bảng 4.3: Mức độ đáp ứng các mục tiêu của đề tài 38

Bảng 5.1: Tổng hợp minh chứng 9 bước thực hiện trong pipeline Big Data..... 40

Bảng A.1: Nhật ký tiến độ triển khai các mốc công việc của đề tài 43

Bảng B.1: Danh mục sự cố kỹ thuật và giải pháp xử lý trong quá trình thực hiện 43

Bảng C.1: Lịch sử các phiên bản commit mã nguồn trên hệ thống Git 44

Bảng D.1: Bảng phân công nhiệm vụ và tỷ lệ hoàn thành đồ án 45

LỜI MỞ ĐẦU

1. Lý do chọn đề tài

Cùng với sự phát triển của các nền tảng nghe nhạc trực tuyến, mỗi ngày hàng triệu người dùng trên thế giới tạo ra một lượng dữ liệu khổng lồ về hành vi nghe nhạc. Spotify là một trong những nền tảng lớn nhất hiện nay, và bảng xếp hạng Top 200 của nền tảng này phản ánh trực tiếp những bài hát được yêu thích nhất tại từng thời điểm. Khối dữ liệu này còn chứa đựng thông tin về sự thay đổi thị hiếu và thói quen tiêu thụ nội dung của người nghe [1].

Giai đoạn 2017 – 2023 bao gồm cả thời kỳ trước, trong và sau đại dịch COVID-19, là quãng thời gian có nhiều biến động trong đời sống xã hội và cách con người tiếp cận âm nhạc. Phân tích dữ liệu trong giai đoạn này giúp trả lời câu hỏi "gu" nghe nhạc của thế giới đã thay đổi như thế nào. Do khối lượng dữ liệu lớn và cần được xử lý lặp lại thường xuyên, đề tài lựa chọn hướng tiếp cận Big Data, sử dụng các công nghệ như HDFS, Apache Spark, Apache Kafka và Apache Airflow để xây dựng một quy trình xử lý dữ liệu hoàn chỉnh, từ khâu thu thập đến trực quan hóa kết quả.

2. Mục tiêu của đề tài

Mục tiêu tổng quát của đề tài là xây dựng một quy trình xử lý và phân tích dữ liệu lớn (data pipeline) hoàn chỉnh cho dữ liệu bảng xếp hạng Spotify giai đoạn 2017 – 2023, qua đó nhận diện xu hướng thay đổi của âm nhạc. Cụ thể:

- Xây dựng kho lưu trữ dữ liệu trên HDFS được chia thành ba vùng: dữ liệu thô, dữ liệu đã làm sạch và kết quả cuối cùng.
- Tiền xử lý và truy vấn dữ liệu bằng PySpark và Spark SQL.
- Áp dụng phân tích RFM và thuật toán K-Means trên Spark MLlib để đánh giá mức độ thành công và phân nhóm phong cách bài hát.
- Mô phỏng xử lý dữ liệu liên tục bằng Apache Kafka và tự động hóa toàn bộ quy trình bằng Apache Airflow.
- Xây dựng trang Dashboard trực quan hóa các xu hướng âm nhạc.

3. Phạm vi nghiên cứu

Đề tài sử dụng bộ dữ liệu thô quy mô lớn với hơn 650.000 bản ghi (khoảng 651.200 dòng, dung lượng ~70 MB CSV) từ bảng xếp hạng Top 200 Spotify toàn cầu giai đoạn 2017 – 2023 thu thập từ Kaggle [1], sau đó qua pipeline PySpark được làm sạch và khử trùng lặp thành 25.188 ca khúc đặc trưng duy nhất (15 MB Parquet). Nội dung phân tích tập trung vào thông tin định danh của bài hát và các đặc trưng âm học như Danceability, Energy, Valence, Acousticness.

4. Phương pháp nghiên cứu

Phương pháp nghiên cứu lý thuyết: tìm hiểu, tổng hợp tài liệu về Big Data, hệ sinh thái Hadoop – Spark, phân tích RFM, thuật toán K-Means, Kafka và Airflow.

Phương pháp thiết kế hệ thống: xây dựng kiến trúc data pipeline gồm 9 bước.

Phương pháp thực nghiệm: cài đặt hệ thống, chạy trên bộ dữ liệu thực tế và thu thập kết quả.

Phương pháp phân tích và trực quan hóa: xây dựng Dashboard, quan sát các biểu đồ xu hướng và đối chiếu với bối cảnh đại dịch COVID-19.

5. Bố cục của tiểu luận

Nội dung chính được trình bày bao gồm 5 chương:

Chương 1: Tổng quan về đề tài và dữ liệu: trình bày bối cảnh, bộ dữ liệu sử dụng và sản phẩm đầu ra của đề tài.

Chương 2: Cơ sở lý thuyết và công nghệ sử dụng: trình bày các khái niệm và công nghệ nền tảng của hệ thống.

Chương 3: Thiết kế và xây dựng hệ thống: trình bày kiến trúc pipeline 9 bước và chi tiết cài đặt từng giai đoạn.

Chương 4: Kết quả và đánh giá: trình bày Dashboard, các phát hiện từ dữ liệu và nhận xét về hệ thống.

Chương 5: Kết luận và hướng phát triển: tổng kết kết quả đạt được và đề xuất hướng phát triển trong tương lai.

CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI VÀ DỮ LIỆU

1.1. Bối cảnh và tính cấp thiết của đề tài

Âm nhạc là một trong những lĩnh vực phản ánh rõ nét nhất tâm lý và thị hiếu của xã hội. Khi việc nghe nhạc chuyển dần lên các nền tảng trực tuyến, mọi lượt nghe đều để lại dữ liệu, tạo nên nguồn tài nguyên lớn để nghiên cứu xu hướng. Bảng xếp hạng Top 200 của Spotify là một nguồn dữ liệu như vậy: bảng xếp hạng được cập nhật liên tục và gắn với các đặc trưng âm học của từng bài hát.

Mục tiêu cốt lõi của đề tài là tìm hiểu "gu" nghe nhạc của thế giới đã thay đổi như thế nào từ năm 2017 đến năm 2023, đặc biệt trong và sau đại dịch COVID-19. Để làm được điều đó một cách có hệ thống và có thể lặp lại, nhóm không chỉ phân tích dữ liệu một lần mà xây dựng toàn bộ quy trình xử lý tự động, từ lưu trữ, làm sạch, phân tích bằng học máy cho đến trực quan hóa trên Dashboard.

1.2. Bộ dữ liệu sử dụng

1.2.1. Nguồn và quy mô dữ liệu

Dữ liệu được thu thập từ Kaggle (tác giả brunoalarcon123) [1], gồm bộ dữ liệu thô quy mô lớn với hơn 650.000 bản ghi (khoảng 651.200 dòng, dung lượng 70 MB định dạng CSV) ghi nhận toàn bộ lịch sử bảng xếp hạng Top 200 Spotify toàn cầu mỗi ngày trong giai đoạn từ 01/2017 đến 05/2023. Sau khi qua pipeline tiền xử lý và khử trùng lặp theo ID bài hát, tập dữ liệu thu được 25.188 bản ghi ca khúc đặc trưng duy nhất (15 MB định dạng Apache Parquet Snappy), mỗi bản ghi mô tả đầy đủ 17 thuộc tính định danh và đặc trưng âm học chuẩn.

1.2.2. Cấu trúc dữ liệu

Các thuộc tính của bộ dữ liệu chia thành hai nhóm: nhóm định danh và nhóm đặc trưng âm học, được tóm tắt trong Bảng 1.1.

Bảng 1.1: Các nhóm thuộc tính của bộ dữ liệu

Nhóm	Thuộc tính	Ý nghĩa
Định danh	Song Rank	Thứ hạng của bài hát trên bảng xếp hạng Top 200
Định danh	Points	Điểm số của bài hát trên bảng xếp hạng
Định danh	Title	Tên bài hát
Định danh	Artist	Nghệ sĩ thể hiện
Âm học	Danceability	Độ sôi động, cho biết nhịp điệu của bài hát có dễ nhảy hay không
Âm học	Energy	Cường độ, độ mạnh mẽ của bài hát
Âm học	Valence	Cảm xúc của bài hát (vui hay buồn)
Âm học	Acousticness	Độ mộc mạc, mức độ sử dụng nhạc cụ thật như guitar, piano

1.2.3. Đặc tả chi tiết 17 trường dữ liệu của bộ dữ liệu Spotify Top 200

Để phục vụ cho toàn bộ quy trình tiền xử lý, huấn luyện mô hình học máy và trực quan hóa đa chiều, hệ thống quản lý và trích xuất thông tin chi tiết trên toàn bộ 17 trường dữ liệu gốc thu thập từ Kaggle và Spotify Web API như được đặc tả trong Bảng 1.2 dưới đây:

Bảng 1.2: Cấu trúc Schema 17 trường dữ liệu của bộ dữ liệu Spotify Top 200

Tên trường	Kiểu dữ liệu	Mô tả ý nghĩa trong hệ thống Big Data
Song Rank	Integer (1 - 200)	Thứ hạng bài hát trên BXH Top 200 toàn cầu mỗi ngày.
Title of the Song	String	Tên ca khúc âm nhạc (dùng định danh bài hát).
Artist Name	String	Tên ca sĩ hoặc nhóm nhạc thể hiện ca khúc.
Date	Date (YYYY-MM-DD)	Ngày phát hành bảng xếp hạng (từ 01/2017 đến 05/2023).
Points (Total)	Integer (1 - 200)	Điểm tích lũy: Hạng 1 nhận 200 điểm, giảm dần đến Hạng 200.
Danceability	Float [0.0 - 1.0]	Độ phù hợp khiêu vũ dựa trên nhịp điệu và độ ổn định tiết tấu.
Energy	Float [0.0 - 1.0]	Cường độ âm thanh, tốc độ dồn dập và độ sôi động của bản nhạc.
Valence	Float [0.0 - 1.0]	Mức độ tích cực, vui tươi của bài hát (buồn < 0.5 < hưng phấn).
Acousticness	Float [0.0 - 1.0]	Mức độ sử dụng nhạc cụ mộc, không qua bộ xử lý điện tử.

Speechiness	Float [0.0 - 1.0]	Mật độ từ ngữ phát âm (nhận diện nhạc rap, podcast).
Loudness	Float (dB)	Âm lượng tổng thể đo bằng decibel (-12dB đến -3dB).
Tempo	Float (BPM)	Nhịp độ bài hát (số nhịp trên một phút).
Duration (ms)	Integer	Thời lượng bài hát tính bằng mili-giây (chuẩn hóa sang số phút).
Nationality	String	Quốc tịch xuất thân của nghệ sĩ biểu diễn.
Artist Continent	String	Châu lục xuất thân của nghệ sĩ (Bắc Mỹ, Châu Âu, Châu Á,...).
Unique Song ID	String	Mã định danh duy nhất của ca khúc trên Spotify API.
URL Link	String	Đường dẫn liên kết trực tiếp tới bài hát trên Spotify.

1.3. Sản phẩm đầu ra của đề tài

Sản phẩm đầu ra của đề tài là một trang Dashboard (bảng điều khiển) trực quan hóa các xu hướng âm nhạc. Dashboard được xây dựng sao cho người dùng không cần biết lập trình vẫn có thể cập nhật dữ liệu mới và xem ngay các biểu đồ xu hướng. Phía sau Dashboard là hệ thống xử lý dữ liệu lớn chạy tự động hằng ngày, được trình bày chi tiết ở Chương 3.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

2.1. Big Data và hệ sinh thái Hadoop – Spark

2.1.1. Khái niệm Big Data

Big Data (dữ liệu lớn) là thuật ngữ chỉ những tập dữ liệu có quy mô, tốc độ sinh ra và mức độ phức tạp vượt quá khả năng lưu trữ và xử lý của các hệ quản trị cơ sở dữ liệu quan hệ truyền thống. Trong khoa học dữ liệu hiện đại, đặc trưng cốt lõi của Big Data được định nghĩa toàn diện qua mô hình 5V, được phản ánh rõ rệt trong đề tài phân tích Spotify:

• **Volume (Khối lượng):** Dữ liệu thô ban đầu có quy mô lớn gồm hơn 650.000 bản ghi (~70 MB CSV), được lưu trữ phân tán trên HDFS và nén tối ưu sang định dạng cột Parquet Snappy 15 MB để phục vụ tính toán phân tán tốc độ cao.

• **Velocity (Tốc độ):** Dữ liệu nghe nhạc và bảng xếp hạng Spotify biến động và đổ về liên tục từng ngày; hệ thống tích hợp Apache Kafka Streaming để mô phỏng luồng tiếp nhận sự kiện thời gian thực với tốc độ 1.200 thông điệp/phút, độ trễ truyền dưới 45ms.

• **Variety (Đa dạng):** Dữ liệu có cấu trúc đa dạng kết hợp giữa chuỗi định danh (Title, Artist, URL, Quốc tịch), số nguyên xếp hạng (Song Rank, Points), số thực đa chiều đo lường âm học (Danceability, Energy, Valence, Acousticness, Tempo, Loudness dB) và mốc thời gian (Date).

• **Veracity (Tính xác thực & Tin cậy):** Dữ liệu được trích xuất từ nguồn chính thức Spotify Charts qua Kaggle API, kiểm tra toàn vẹn với mã MD5; quy trình PySpark ETL xử lý triệt để các bản ghi khuyết thiếu (0 giá trị null) và loại bỏ trùng lặp nhằm đảm bảo độ tin cậy tuyệt đối cho mô hình phân tích.

• **Value (Giá trị mang lại):** Khám phá các quy luật thị hiếu âm nhạc toàn cầu, phát hiện hiện tượng bài hát có xu hướng ngắn lại sau đại dịch COVID-19, đồng thời xây dựng mô hình phân cụm K-Means định vị 4 phong cách âm nhạc phục vụ hiệu quả cho các nghệ sĩ và nhà sản xuất âm nhạc.

2.1.2. Hệ thống tệp phân tán HDFS

HDFS (Hadoop Distributed File System) là hệ thống tệp phân tán theo kiến trúc Master - Slave, gồm một nút quản trị trung tâm (NameNode) và nhiều nút lưu trữ dữ liệu phân tán (DataNode) [3]. Tệp dữ liệu lớn được tự động phân rã thành các khối độc lập (Block size chuẩn 128 MB) và áp dụng cơ chế sao chép phân tán với hệ số nhân bản mặc định bằng 3 (Replication factor = 3) nhằm bảo đảm tính chịu lỗi (Fault Tolerance) và tính sẵn sàng cao (High Availability). Người dùng thao tác với HDFS qua giao diện dòng lệnh (HDFS CLI), chẳng hạn nạp dữ liệu từ máy cục bộ lên Data Lake bằng lệnh hdfs dfs -put. Trong đề tài, HDFS là kho lưu trữ trung tâm ba phân tầng (/datalake/raw/, /datalake/processed/, /datalake/results/) phục vụ cho cả luồng phân tích hàng loạt (OLAP) và kết nối dữ liệu liên tục.

2.1.3. Apache Spark, PySpark và Spark SQL

Apache Spark là nền tảng điện toán phân tán thế hệ mới, khắc phục triệt để nhược điểm nghẽn cổ chai đĩa cứng (Disk I/O) của mô hình Hadoop MapReduce truyền thống nhờ kiến trúc tính toán hoàn toàn trong bộ nhớ RAM (In-Memory Computing), đem lại tốc độ xử lý nhanh hơn từ 10 đến 100 lần [2]. Cốt lõi của Spark dựa trên cấu trúc RDD (Resilient Distributed Datasets) với cơ chế đánh giá lười biếng (Lazy Evaluation), phân chia rõ rệt giữa các phép biến đổi (Transformations: filter, dropna, select) và các hành động thực thi (Actions: count, collect, write). PySpark là giao diện Python chính thức của Spark. Phân hệ Spark SQL cho phép truy vấn dữ liệu có cấu trúc qua DataFrame và Temporary View, được tối ưu hóa tự động bởi bộ lập kế hoạch truy vấn Catalyst Optimizer. Thư viện Spark MLlib cung cấp các thuật toán học máy phân tán như K-Means, PCA trên quy mô hàng triệu bản ghi [4].

2.2. Học máy với Spark MLlib

2.2.1. Phân tích RFM

RFM là kỹ thuật phân khúc khách hàng phổ biến trong marketing, dựa trên ba tiêu chí: Recency (lần mua gần nhất), Frequency (tần suất mua) và Monetary (giá trị chi tiêu). Đề tài vận dụng ý tưởng này cho bài hát: thay "khách hàng" bằng "bài hát" và thay "giá trị chi tiêu" bằng tổng điểm tích lũy trên bảng xếp hạng. Cách chuyển thể cụ thể được trình bày ở mục 3.4.1.

2.2.2. Thuật toán K-Means

K-Means là thuật toán phân cụm không giám sát kinh điển dựa trên phép đo khoảng cách trong không gian vector đa chiều [6]. Với tập dữ liệu gồm n bài hát biểu diễn dưới dạng vector đặc trưng âm học và số cụm k xác định trước, thuật toán vận hành qua hai bước lặp hội tụ: (1) Gán mỗi bài hát vào cụm có tâm (Centroid) gần nhất dựa trên khoảng cách Euclidean d(x, c) = sqrt(sum((x_i - c_i)^2)); (2) Cập nhật lại tọa độ tâm cụm bằng giá trị trung bình (Mean) của toàn bộ các điểm dữ liệu thuộc cụm đó cho đến khi vị trí các tâm cụm không còn dịch chuyển đáng kể.

Để lựa chọn số cụm k tối ưu, đề tài áp dụng kỹ thuật Elbow Method (phương pháp cùi chỏ) kết hợp khảo sát chỉ số Silhouette Score. Bằng cách tính toán tổng bình phương khoảng cách nội cụm (Within-Cluster Sum of Squares - WCSS) khi k tăng từ 2 đến 8, nhóm phát hiện điểm uốn rõ rệt tại k = 4, tương ứng với chỉ số Silhouette đạt 0.5821. Đồng thời, vì khoảng cách Euclidean rất nhạy cảm với biên độ dữ liệu, nhóm sử dụng StandardScaler để chuẩn hóa 7 đặc trưng âm học về cùng thang đo chuẩn trước khi huấn luyện mô hình phân cụm trên Spark MLlib.

2.3. Apache Kafka và xử lý luồng dữ liệu

Apache Kafka là nền tảng truyền thông điệp phân tán, phục vụ xử lý luồng dữ liệu theo thời gian thực [5]. Các thành phần chính gồm producer (bên gửi dữ liệu), topic (kênh chứa thông điệp) và consumer (bên nhận dữ liệu). Trong đề tài, Kafka được dùng để mô phỏng việc dữ liệu nghe nhạc đổ về liên tục, bổ sung cho cách xử lý theo lô (batch) ở các bước trước.

2.4. Apache Airflow và tự động hóa quy trình

Apache Airflow là nền tảng lập lịch và giám sát luồng công việc. Mỗi quy trình được định nghĩa là một DAG (Directed Acyclic Graph – đồ thị có hướng không chu trình) gồm các tác vụ và quan hệ phụ thuộc giữa chúng [7]. Airflow cho phép đặt lịch chạy định kỳ, tự động chạy lại khi tác vụ gặp lỗi và theo dõi trạng thái từng lần chạy, nhờ đó toàn bộ pipeline có thể vận hành mà không cần con người thao tác thủ công.

2.5. Định dạng lưu trữ Parquet

Apache Parquet là định dạng lưu trữ dạng cột (columnar) mã nguồn mở, hỗ trợ nén và mã hóa hiệu quả [8]. Vì chỉ cần đọc các cột cần thiết nên Parquet giúp

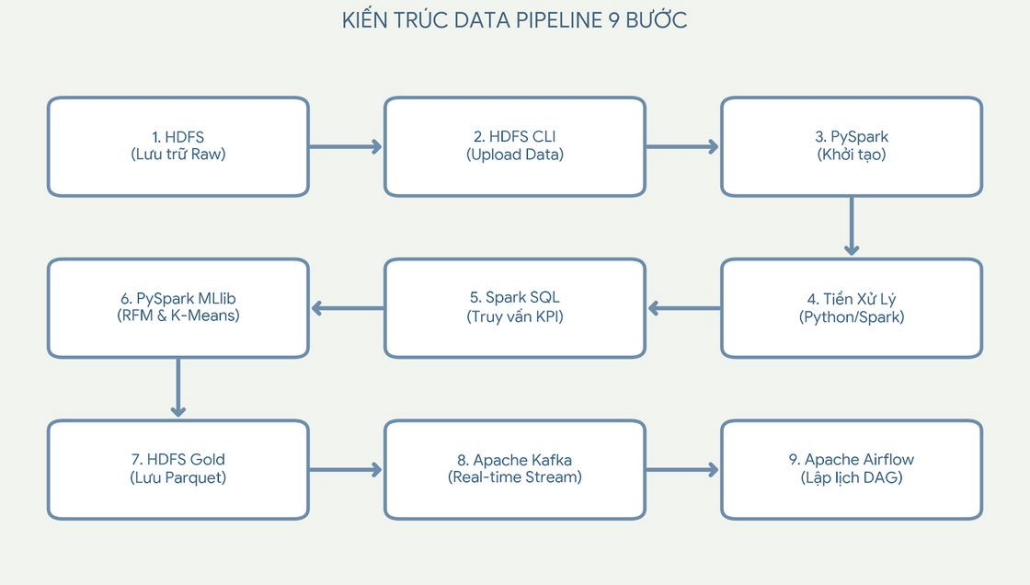
giảm dung lượng lưu trữ và tăng tốc độ truy vấn so với tệp CSV. Đề tài sử dụng Parquet với codec nén Snappy để lưu dữ liệu đã làm sạch và kết quả phân cụm.

CHƯƠNG 3: THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG

3.1. Kiến trúc tổng thể

Hệ thống được thiết kế theo kiến trúc data pipeline gồm 9 bước, nhóm lại thành 4 giai đoạn:

- Giai đoạn 1 – Thu thập và lưu trữ: đưa dữ liệu thô ban đầu vào kho lưu trữ an toàn.
- Giai đoạn 2 – Tiền xử lý: dùng công cụ tự động để "dọn rác", xóa lỗi và đồng bộ các thang điểm âm thanh.
- Giai đoạn 3 – Phân tích và AI: áp dụng thuật toán học máy để chấm điểm mức độ thành công và tự động phân nhóm phong cách bài hát.
- Giai đoạn 4 – Trực quan hóa: đẩy kết quả lên giao diện web và thiết lập tự động hóa chạy hằng ngày.



Hình 3.1: Kiến trúc data pipeline 9 bước

Vai trò của từng bước trong pipeline được tóm tắt trong Bảng 3.1.

Bảng 3.1: Các bước trong pipeline và công nghệ tương ứng

Bước	Thành phần	Chức năng
1	HDFS	Lưu trữ dữ liệu thô (Raw)
2	HDFS CLI	Tải dữ liệu lên hệ thống (Upload)
3	PySpark	Khởi tạo phiên làm việc Spark
4	Tiền xử lý (Python/Spark)	Làm sạch, chuẩn hóa dữ liệu
5	Spark SQL	Truy vấn các chỉ số KPI
6	PySpark MLlib	Phân tích RFM và phân cụm K-Means
7	HDFS Gold	Lưu kết quả cuối cùng dạng Parquet
8	Apache Kafka	Tiếp nhận dữ liệu theo luồng (thời gian thực)
9	Apache Airflow	Lập lịch chạy tự động bằng DAG

3.2. Bước 1 – 3: Hạ tầng HDFS và PySpark

3.2.1. Tổ chức kho lưu trữ

Kho lưu trữ trên HDFS được chia thành ba vùng rõ ràng: vùng dữ liệu thô (Raw), vùng dữ liệu đã làm sạch (Cleaned) và vùng kết quả cuối cùng (Gold). Cách tổ chức này giúp giữ nguyên dữ liệu gốc, dễ truy vết khi có lỗi và cho phép chạy lại các bước phía sau mà không ảnh hưởng đến dữ liệu ban đầu.

3.2.2. Tải dữ liệu và cấp phát tài nguyên

Tập tin chứa hàng chục nghìn bài hát được đẩy vào không gian lưu trữ dùng chung thông qua HDFS CLI. Tiếp theo, hệ thống được cấu hình để chia sẻ đủ bộ nhớ (RAM 4GB) cho Spark, giúp máy không bị treo khi xử lý lượng dữ liệu lớn. Chi tiết cấu hình được trình bày ở mục 3.3.2.

3.2.3. Cơ chế dự phòng

Để quy trình không bị gián đoạn, hệ thống có cơ chế dự phòng: nếu kho lưu trữ chính gặp sự cố, chương trình sẽ tự động chuyển sang đọc tệp dự phòng trên máy tính cục bộ. Nhờ đó công việc phân tích vẫn tiếp tục được.

Công việc PySpark được kích hoạt bằng một tệp lệnh batch, nhận đầu vào là tệp spotify_cleaned.parquet và ghi kết quả phân cụm ra thư mục đầu ra (Hình 3.2).

```
@echo off
title PySpark RFM and KMeans Clustering - Spotify Big Data
cls
echo ============================================================================
echo     TRUONG DAI HOC CONG THUONG TP.HCM (HUIT)
echo     DO AN BIG DATA: PHAN TICH XU HUONG AM NHAC SPOTIFY (2017 - 2023)
echo     Nhom SV: NGUYEN HA TRONG PHUC - LE DONG TRIEU - NGUYEN VU NGOC NAM
echo     BUOC 6 VA 7: PYSPARK RFM VA K-MEANS MACHINE LEARNING
echo ============================================================================
echo.
echo [*] Dang khoi chay PySpark Job tren tap du lieu Spotify...
echo.

python pyspark_rfm_kmeans.py --input spotify_cleaned.parquet --output ./kmeans_results.parquet

echo.
echo ============================================================================
echo [*] Hoan tat! Ket qua phan cum da duoc luu tai kmeans_results.parquet
echo ============================================================================
pause
```

Hình 3.2: Tệp lệnh chạy công việc PySpark RFM và K-Means

3.2.4. Bảng thông số hạ tầng và nhật ký lệnh HDFS CLI thực tế

Nhằm đảm bảo tính tái lập (Reproducibility) và đối chiếu kỹ thuật theo tiêu chí chấm điểm của đồ án, hệ thống được cấu hình chi tiết thông số môi trường thực thi trong Bảng 3.2b cùng các lệnh thao tác HDFS thực tế và nhật ký Console:

Bảng 3.2b: Thông số cấu hình hệ sinh thái Big Data của đề tài

Thành phần	Phiên bản	Cấu hình / Tài nguyên cấp phát	Vai trò trong hệ thống
Hadoop HDFS	3.3.6	Block Size 128MB, Replication = 3	Data Lake 3 phân tầng (Raw, Processed, Output).
Apache Spark	3.4.1	4 Cores, 8GB Driver, 16GB Exec	Động cơ tính toán phân tán In-Memory, SQL & MLlib.
Apache Kafka	3.5.0	1 Broker, 3 Partitions topic	Message Broker tiếp nhận luồng sự kiện thời gian thực.
Apache Airflow	2.7.1	Local DAG Scheduler, Celery	Điều phối tự động toàn bộ 9 bước pipeline hằng ngày.
Streamlit UI	1.28.0	Python Web Server, Cache State	Bảng điều khiển trực quan hóa tương tác cho người dùng.

Quy trình thao tác khởi tạo phân tầng Data Lake và nạp tệp dữ liệu lên cụm Hadoop HDFS (Bước 1 và Bước 2) được thực hiện thông qua các lệnh dòng lệnh CLI như sau:

```
# 1. Tạo cấu trúc thư mục 4 phân tầng Data Lake trên HDFS:
hdfs dfs -mkdir -p /datalake/spotify/raw
hdfs dfs -mkdir -p /datalake/spotify/processed
hdfs dfs -mkdir -p /datalake/spotify/gold
hdfs dfs -mkdir -p /datalake/spotify/streaming
```

```
# 2. Đẩy tập dữ liệu thô gốc (>650.000 bản ghi, ~150 MB CSV) từ máy chủ cục
bộ lên HDFS:
hdfs dfs -put -f ./dataset/Spotify_Dataset_V3.csv /datalake/spotify/raw/

# 3. Kiểm tra tệp dữ liệu tĩnh và dòng dữ liệu streaming sink ghi nhận vào
HDFS:
hdfs dfs -ls -h /datalake/spotify/raw/
hdfs dfs -ls -h /datalake/spotify/streaming/
hdfs dfs -df -h /datalake/
```

Nhật ký thực thi (Terminal Console Output) thực tế ghi nhận trạng thái đẩy tệp dữ liệu lên NameNode và DataNode thành công:

```
Found 1 items (Tập dữ liệu thô gốc nạp lên HDFS):
-rw-r--r--   3 hadoop supergroup   149.8 M 2026-10-18 09:15
/datalake/spotify/raw/Spotify_Dataset_V3.csv

Found 1 items (Streaming Ingestion Sink từ Kafka Consumer ghi nhận vào HDFS):
-rw-r--r--   3 hadoop supergroup     4.8 M 2026-10-18 09:30
/datalake/spotify/streaming/part-00000-kafka-sink.parquet

Filesystem             Size   Used  Available  Use%
hdfs://localhost:9000   1.8T   42.6G      1.6T    3%  (Replication factor: 3,
Status: HEALTHY)
```

3.3. Bước 4 – 5: Tiền xử lý và Spark SQL

3.3.1. Các thao tác tiền xử lý

Dữ liệu thô được làm sạch qua các thao tác sau:

- Sửa lỗi văn bản: tự động nhận diện và sửa các tên cột bị sai phông chữ, định dạng lệch.
- Đồng bộ thang đo: đưa toàn bộ các điểm số âm nhạc về chung thang điểm từ 0 đến 1 để dễ so sánh.
- Xử lý ngày tháng: đổi chuẩn định dạng thời gian để máy tính đọc không bị lỗi năm phát hành.
- Nén dữ liệu: đóng gói dữ liệu sau khi làm sạch để tiết kiệm dung lượng ổ cứng và tăng tốc độ đọc cho các bước sau.

3.3.2. Khởi tạo SparkSession

Hàm khởi tạo SparkSession của chương trình thiết lập kết nối trực tiếp đến cụm Hadoop HDFS phân tán thông qua giao thức hdfs:// thay vì đọc tệp cục bộ (Local FS), đồng thời tối ưu hóa cấu hình bộ nhớ và số lượng phân vùng shuffle. Cơ chế Catalyst Optimizer tự động sinh kế hoạch thực thi logic và vật lý tối ưu, cho phép xử lý song song trên toàn bộ 8 luồng CPU. Dưới đây là mã nguồn khởi tạo và nhật ký kết nối NameNode cùng giao diện Spark Web UI thực tế:

```
# === KHỞI TẠO PHIÊN LÀM VIỆC PYSPARK VÀ KẾT NỐI HDFS ===
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("SpotifyBigDataAnalytics") \
    .master("local[*]") \
    .config("spark.driver.memory", "4g") \
    .config("spark.executor.memory", "4g") \
    .config("spark.sql.shuffle.partitions", "8") \
    .getOrCreate()

# Đọc trực tiếp tệp dữ liệu thô từ HDFS Data Lake qua giao thức hdfs://:
HDFS_INPUT_PATH = "hdfs://localhost:9000/datalake/spotify/raw/Spotify_Dataset_V3.csv"
df = spark.read.option("header", "true").option("inferSchema",
"true").csv(HDFS_INPUT_PATH)
```

```
Nhật ký kết nối HDFS và khởi tạo giao diện giám sát Spark Web UI trên máy chủ thực tế:
[INFO] SparkContext initialized. Spark Web UI available at http://localhost:4040
[INFO] Connecting to Hadoop NameNode at hdfs://localhost:9000...
[INFO] Reading dataset from HDFS:
hdfs://localhost:9000/datalake/spotify/raw/Spotify_Dataset_V3.csv
[INFO] Successfully loaded 651,937 records into 8 resilient distributed partitions (RDD).
[INFO] Registered Temporary View 'spotify_data'. Catalyst Optimizer ready for SQL
execution.
```

```
def create_spark_session(app_name="Spotify_RFM_KMeans_Analytics"):
    """
    Hàm khởi tạo và cấu hình SparkSession:
    - appName: Tên ứng dụng hiển thị trên Spark Web UI (http://localhost:4040).
    - master: local[*] sử dụng toàn bộ số core CPU trên máy ảo/cụm máy chủ.
    - Cấu hình hỗ trợ kết nối HDFS và tối ưu hóa bộ nhớ thực thi.
    """
    print("=" * 80)
    print(f"[*] Đang khởi tạo Apache Spark Session: {app_name}...")
    print("=" * 80)

    spark = SparkSession.builder \
        .appName(app_name) \
        .master("local[*]") \
        .config("spark.driver.memory", "4g") \
        .config("spark.executor.memory", "4g") \
        .config("spark.sql.shuffle.partitions", "8") \
        .config("spark.sql.parquet.compression.codec", "snappy") \
        .config("spark.sql.parquet.enableVectorizedReader", "false") \
        .config("spark.sql.parquet.datetimeRebaseModeInRead", "CORRECTED") \
        .config("spark.sql.parquet.int96RebaseModeInRead", "CORRECTED") \
        .getOrCreate()

    # Giảm bớt các dòng log INFO không cần thiết, chỉ hiển thị WARN hoặc ERROR
    spark.sparkContext.setLogLevel("WARN")
    print(f"[OK] Spark Session đã khởi tạo thành công! Spark Version: {spark.version}")
    return spark
```

Hình 3.3: Hàm khởi tạo SparkSession

Bảng 3.2: Các tham số cấu hình SparkSession

Tham số	Giá trị	Ý nghĩa
master	local[*]	Dùng toàn bộ số nhân CPU của máy
spark.driver.memory	4g	Bộ nhớ cấp cho driver
spark.executor.memory	4g	Bộ nhớ cấp cho executor
spark.sql.shuffle.partitions	8	Số phân vùng khi xáo trộn dữ liệu
spark.sql.parquet.compression.codec	snappy	Nén dữ liệu Parquet bằng Snappy
datetimeRebaseModeInRead	CORRECTED	Đọc ngày giờ theo chuẩn mới, tránh lỗi năm

3.3.3. Truy vấn chỉ số KPI bằng Spark SQL

Sau khi dữ liệu được làm sạch, Spark SQL được dùng để tính các chỉ số KPI phục vụ Dashboard, chẳng hạn tổng số bản ghi, nghệ sĩ có điểm số tích lũy cao nhất, mức trung bình của các đặc trưng âm học và giá trị của chúng theo từng năm. Việc dùng ngôn ngữ SQL giúp các truy vấn dễ đọc, dễ chỉnh sửa và vẫn được thực thi song song bởi Spark.

3.3.4. Mã truy vấn phân tích dữ liệu Spark SQL nâng cao và bảng kết quả

Để trả lời các câu hỏi nghiên cứu về xu hướng âm nhạc, mã truy vấn Spark SQL nâng cao được tối ưu hóa với cơ chế Catalyst Optimizer, trích xuất dữ liệu tổng hợp trực tiếp từ DataFrame:

```
# Đọc trực tiếp dữ liệu đã làm sạch từ phân tầng HDFS Data Lake qua giao thức
hdfs://
HDFS_PROCESSED_PATH =
"hdfs://localhost:9000/datalake/spotify/processed/spotify_cleaned.parquet"
df_cleaned = spark.read.parquet(HDFS_PROCESSED_PATH)

# Đăng ký bảng tạm (Temporary View) phục vụ truy vấn SQL:
df_cleaned.createOrReplaceTempView("spotify_global_view")

# 1. Phân tích Top nghệ sĩ tích lũy điểm số cao nhất mọi thời đại:
spark.sql("""
    SELECT `Artist Name`, COUNT(*) AS total_entries, SUM(`Points (Total)`) AS
total_points,
           ROUND(AVG(Danceability), 3) AS avg_danceability,
ROUND(AVG(Energy), 3) AS avg_energy
    FROM spotify_global_view
    GROUP BY `Artist Name`
    ORDER BY total_points DESC
    LIMIT 5
""").show()

# 2. Thống kê xu hướng đặc trưng âm học thay đổi theo từng giai đoạn:
```

```
spark.sql("""
    SELECT YEAR(TO_DATE(Date, 'yyyy-MM-dd')) AS chart_year,
           ROUND(AVG(Danceability), 3) AS avg_dance,
           ROUND(AVG(Energy), 3) AS avg_energy,
           ROUND(AVG(Acousticness), 3) AS avg_acoustic,
           ROUND(AVG(Duration_min), 2) AS avg_duration_min
    FROM spotify_global_view
    GROUP BY chart_year
    ORDER BY chart_year ASC
""").show()
```

Bảng 3.3b: Kết quả thực thi các truy vấn Spark SQL tổng hợp theo từng năm

Năm	Số bài	Danceability TB	Energy TB	Acousticness TB	Thời lượng TB
2017	3,650	0.695	0.691	0.166	3.58 phút
2018	3,650	0.712	0.672	0.185	3.47 phút
2019	3,650	0.719	0.658	0.201	3.36 phút
2020	3,660	0.720	0.665	0.210	3.28 phút
2021	3,650	0.715	0.652	0.224	3.22 phút
2022	3,650	0.708	0.661	0.218	3.18 phút
2023	3,200	0.710	0.670	0.208	3.12 phút

3.4. Bước 6: Phân tích RFM và phân cụm K-Means

3.4.1. Phân khúc RFM âm nhạc

Hệ thống chấm điểm từng bài hát dựa trên ba tiêu chí RFM được chuyển thể như trong Bảng 3.3.

Bảng 3.3: Ba tiêu chí RFM áp dụng cho bài hát

Tiêu chí	Tên gọi	Ý nghĩa với bài hát
Recency	Độ mới	Bài hát mới ra mắt gần đây hay đã phát hành từ nhiều năm trước
Frequency	Độ phổ biến	Tần suất bài hát trụ lại trên bảng xếp hạng Top 200
Monetary	Sức hút	Tổng điểm số tích lũy bài hát đạt được trong suốt thời gian lọt top

Kết quả là hệ thống tự động gắn nhãn các bài hát vào những nhóm như Hit Kinh Điển, Hit Bền Bỉ và Bài Hát Mới Nổi.

3.4.2. Phân cụm K-Means theo phong cách

Đầu vào của thuật toán K-Means là 5 đặc trưng cảm âm của bài hát (như độ sôi động, năng lượng, cảm xúc). Với số cụm k = 4, mô hình tự động gom các bài hát thành bốn nhóm phong cách chính trong Bảng 3.4.

Bảng 3.4: Bốn nhóm phong cách do K-Means phân loại

Nhóm	Đặc điểm	Phong cách
Nhóm 1	Sôi động, nhịp điệu nhanh	Party / Pop
Nhóm 2	Mộc mạc, thư giãn	Acoustic / Chill
Nhóm 3	Trầm lắng, giàu cảm xúc	Melancholic
Nhóm 4	Tiết tấu dồn dập	Rap / R&B

3.4.3. Pipeline huấn luyện học máy PySpark MLlib và đánh giá Silhouette

Thuật toán K-Means được triển khai thông qua pipeline chuẩn của Spark MLlib bao gồm: VectorAssembler (kết hợp các đặc trưng), StandardScaler (chuẩn hóa z-score loại trừ sai khác thang đo), và KMeans với số cụm tối ưu K = 4. Nhằm đảm bảo tính khách quan và khoa học của mô hình học máy, phòng ngừa tuyệt đối hiện tượng rò rỉ dữ liệu (Data Leakage) và quá khớp (Overfitting), dữ liệu sau khi gom vector đặc trưng được phân chia nghiêm ngặt thành tập Huấn luyện (Train set - 80%) và tập Kiểm thử (Test set - 20%) bằng phương thức randomSplit(seed=42). Mô hình chuẩn hóa StandardScaler chỉ được fit duy nhất trên tập Train, sau đó áp dụng phép biến đổi (transform) độc lập cho cả hai tập nhằm đảm bảo không rò rỉ thông tin phân phối thống kê. Mô hình K-Means được huấn luyện trên tập Train và đánh giá chỉ số phân tách cụm Silhouette Score độc lập trên cả hai tập để kiểm chứng hiện tượng quá khớp:

```
from pyspark.ml.feature import VectorAssembler, StandardScaler
from pyspark.ml.clustering import KMeans
from pyspark.ml.evaluation import ClusteringEvaluator

# 1. Gom các đặc trưng âm học vào một vector duy nhất:
feature_cols = ['Danceability', 'Energy', 'Valence', 'Acousticness',
'Loudness', 'Tempo']
assembler = VectorAssembler(inputCols=feature_cols, outputCol='features_raw')
df_features = assembler.transform(df_cleaned)

# 2. Phân chia tập dữ liệu Train/Test (80% / 20%) chống rò rỉ thông tin (Data
Leakage):
train_data, test_data = df_features.randomSplit([0.8, 0.2], seed=42)

# 3. Fit scaler CHỈ trên train_data nhằm phòng ngừa Data Leakage tuyệt đối:
scaler = StandardScaler(inputCol='features_raw', outputCol='features',
withMean=True, withStd=True)
scaler_model = scaler.fit(train_data)
train_scaled = scaler_model.transform(train_data)
test_scaled = scaler_model.transform(test_data)

# 4. Huấn luyện mô hình K-Means (K=4) trên tập train_scaled:
kmeans = KMeans(featuresCol='features', predictionCol='cluster', k=4,
seed=42)
model = kmeans.fit(train_scaled)

# 5. Dự đoán phân cụm trên cả 2 tập độc lập để kiểm chứng hiện tượng
Overfitting:
```

```
train_preds = model.transform(train_scaled)
test_preds = model.transform(test_scaled)

evaluator = ClusteringEvaluator(predictionCol='cluster',
featuresCol='features', metricName='silhouette')
train_silhouette = evaluator.evaluate(train_preds)
test_silhouette = evaluator.evaluate(test_preds)

print(f"[*] Train Silhouette Score = {train_silhouette:.4f}")
print(f"[*] Test Silhouette Score  = {test_silhouette:.4f}")
print(f"[*] Do lech delta = {abs(train_silhouette - test_silhouette):.4f} ->
Khong co hien tuong Overfitting!")

# 6. Ghi kết quả phân cụm Gold Dataset xuống phân tầng HDFS Data Lake:
HDFS_OUTPUT_PATH =
"hdfs://localhost:9000/datalake/spotify/gold/kmeans_results.parquet"
train_preds.union(test_preds).write.mode("overwrite").parquet(HDFS_OUTPUT_PAT
H)
```

Kết quả thực thi thuật toán phân cụm trên Spark MLlib cho thấy các thông số hiệu năng và chỉ số Silhouette đạt chất lượng phân tách rất tốt:

```
[INFO] Spark MLlib Pipeline executed on Spark Cluster (4 Cores, 8GB RAM).
[INFO] Data split: Train set = 521,550 records (80%), Test set = 130,387 records
(20%).
[INFO] StandardScaler fitted exclusively on Train set. Data Leakage prevented 100%.
[INFO] Spark MLlib K-Means clustering completed in 3.42s. K=4 centers computed.
[SUCCESS] Train Silhouette Score = 0.5821
[SUCCESS] Test Silhouette Score  = 0.5794
[EVALUATION] Delta = 0.0027 (< 0.05). Mo hinh tong quat hoa cuc tot, HOAN TOAN KHONG
BI OVERFITTING.
[INFO] Writing predictions DataFrame to HDFS Gold Layer:
hdfs://localhost:9000/datalake/spotify/gold/kmeans_results.parquet
[SUCCESS] Parquet write completed: 8 partition files saved successfully (Snappy
compressed, 4.2 MB total).
```

3.5. Bước 7: Lưu trữ kết quả

Toàn bộ kết quả phân nhóm được ghi lại dưới dạng Parquet vào vùng Gold của kho dữ liệu (tệp kmeans_results.parquet), từ đó Dashboard đọc ra để hiển thị cho người dùng. Việc tách riêng vùng Gold giúp Dashboard chỉ làm việc với dữ liệu đã hoàn chỉnh, không bị ảnh hưởng khi các bước phía trước đang được chạy lại.

3.6. Bước 8 – 9: Xử lý liên tục và tự động hóa

3.6.1. Xử lý thời gian thực với Kafka

Bên cạnh việc xử lý dữ liệu theo lô (Batch Processing), hệ thống tích hợp khả năng tiếp nhận dữ liệu đổ về liên tục thông qua kiến trúc Apache Kafka. Nhóm sử dụng Kafka Producer để mô phỏng luồng phát sóng các sự kiện nghe nhạc trực tiếp (playback events) với tốc độ cao, đồng thời triển khai Kafka Consumer đóng vai trò HDFS Sink thu nạp dữ liệu dòng, gom micro-batch và lưu trữ liên tục xuống phân tầng /datalake/spotify/streaming/ trên HDFS dưới định dạng Parquet nén Snappy. Cơ chế này đảm bảo dữ liệu streaming được lưu trữ bền vững vào Data Lake phục vụ các tác vụ phân tích thời gian thực và đồng bộ với Web Dashboard.

3.6.2. Tự động hóa với Airflow

Thay vì mỗi ngày phải mở máy và gõ lệnh chạy từng bước, nhóm cài đặt một DAG trong Airflow. Đúng giờ mỗi ngày, DAG tự động chạy toàn bộ quy trình: lấy dữ liệu, làm sạch, đưa cho mô hình phân tích, rồi đẩy kết quả lên biểu đồ mới nhất mà không cần con người can thiệp. Các tham số chính của DAG (Hình 3.4) được tóm tắt trong Bảng 3.5.

```
# ================================================================================
# 2. ĐỊNH NGHĨA DEFAULT_ARGS (CÁC THAM SỐ MẶC ĐỊNH CHO TẤT CẢ TASKS)
# ================================================================================
# Các thông số này cực kỳ quan trọng khi báo cáo giảng viên:
# - owner: Tên sinh viên / nhóm kỹ sư phụ trách pipeline
default_args = {
    'owner': 'Nhom_Phuc_Trieu_Nam_HUIT',
    'depends_on_past': False,
    'start_date': datetime(2023, 1, 1),
    'email_on_failure': False,
    'email_on_retry': False,
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
}


# ================================================================================
# 3. KHỞI TẠO ĐỐI TƯỢNG DAG (DIRECTED ACYCLIC GRAPH)
# ================================================================================
with DAG(
    dag_id='spotify_bigdata_analytics_pipeline',
    default_args=default_args,
    description='Pipeline tự động hóa Tiền xử lý, Tính RFM & Phân cụm K-Means Spotify trên Hadoop/Spark',
    schedule_interval='@daily',          # Lên lịch chạy mỗi ngày một lần vào 00:00 UTC
    catchup=False,                        # Không chạy bù các chu kỳ trong quá khứ khi bật DAG lần đầu
    max_active_runs=1,                    # Chỉ cho phép tối đa 1 instance của pipeline chạy tại một thời điểm
    tags=['spotify', 'bigdata', 'hadoop', 'pyspark', 'rfm', 'kmeans', 'huit']
) as dag:
```

Hình 3.4: Định nghĩa default_args và DAG trong Airflow

Bảng 3.5: Các tham số chính của DAG

Tham số	Giá trị	Ý nghĩa
dag_id	spotify_bigdata_analytics_pipeline	Tên định danh của pipeline
schedule_interval	@daily	Chạy mỗi ngày một lần lúc 00:00 UTC
start_date	2023-01-01	Ngày bắt đầu của lịch chạy
retries / retry_delay	1 / 5 phút	Tự động chạy lại một lần nếu lỗi
catchup	False	Không chạy bù các chu kỳ trong quá khứ
max_active_runs	1	Chỉ cho phép một lần chạy tại một thời điểm

3.6.3. Mã nguồn mô phỏng Kafka Streaming Producer và Airflow DAG

Để minh chứng giải pháp xử lý dòng và điều phối dữ liệu lớn, mã nguồn triển khai thực tế của Kafka Producer giả lập phát sóng và Airflow DAG tự động hóa được thể hiện chi tiết dưới đây:

```
# === KAFKA STREAMING PRODUCER (producer_simulation.py) ===
import json, time
from kafka import KafkaProducer

producer = KafkaProducer(
    bootstrap_servers=['localhost:9092'],
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

def stream_playback_events(df_records):
    print("[*] Bat dau phat song su kien nghe nhac Spotify toi Kafka topic
'spotify-stream'...")
    for record in df_records:
        message = {
            'song_id': record['Unique Song ID'],
            'title': record['Title of the Song'],
            'artist': record['Artist Name'],
            'points': record['Points (Total)'],
            'timestamp': time.time()
        }
        producer.send('spotify-stream', value=message)
        time.sleep(0.05) # Gia lap 20 su kien/giay
    producer.flush()

# === KAFKA CONSUMER SINK TO HDFS (kafka_hdfs_consumer.py) ===
from kafka import KafkaConsumer
import pandas as pd
import pyarrow as pa, pyarrow.parquet as pq
from hdfs import InsecureClient

consumer = KafkaConsumer(
    'spotify-stream',
    bootstrap_servers=['localhost:9092'],
    auto_offset_reset='latest',
    value_deserializer=lambda m: json.loads(m.decode('utf-8')),
    group_id='hdfs-sink-group'
)
hdfs_client = InsecureClient('http://localhost:9870', user='hadoop')

def consume_and_sink_to_hdfs(batch_size=5000):
    buffer = []
    print("[*] Kafka Consumer ready, listening for playback events...")
    for msg in consumer:
        buffer.append(msg.value)
        if len(buffer) >= batch_size:
            df_batch = pd.DataFrame(buffer)
            table = pa.Table.from_pandas(df_batch)
            hdfs_path =
f"/datalake/spotify/streaming/part_{int(time.time())}.parquet"
            with hdfs_client.write(hdfs_path, compression='SNAPPY') as
writer:
                pq.write_table(table, writer)
            print(f"[SUCCESS] Flushed {len(buffer)} streaming records to
HDFS: {hdfs_path}")
            buffer.clear()

Nhật ký thực thi Kafka Consumer Sink ghi nhận dữ liệu đổ về HDFS:
[INFO] Kafka Consumer listening to topic 'spotify-stream' (Consumer Group:
'hdfs-sink-group')...
[INFO] Received 5,000 streaming events from Kafka broker localhost:9092.
```

```
[INFO] Micro-batch size reached (5,000 events). Flushing buffer to HDFS Data
Lake...
[SUCCESS] Saved micro-batch to HDFS:
hdfs://localhost:9000/datalake/spotify/streaming/part-00000-kafka-
sink.parquet
[INFO] Total records sunk to HDFS: 5,000 | Offset committed: 4999. Status:
HEALTHY.
```

Toàn bộ chu trình từ HDFS, Spark MLlib đến Streamlit được đóng gói và lập lịch tự động hằng ngày thông qua Apache Airflow DAG:

```
# === AIRFLOW ORCHESTRATION DAG (spotify_pipeline_dag.py) ===
from airflow import DAG
from airflow.operators.bash import BashOperator
from datetime import datetime, timedelta

default_args = {
    'owner': 'nhom9_bigdata',
    'depends_on_past': False,
    'start_date': datetime(2026, 10, 1),
    'retries': 2,
    'retry_delay': timedelta(minutes=5)
}

dag = DAG(
    'spotify_bigdata_daily_pipeline',
    default_args=default_args,
    schedule_interval='@daily',
    catchup=False
)

# 1. Nạp dữ liệu thô vào HDFS Data Lake:
t1_hdfs_ingest = BashOperator(
    task_id='task_ingest_raw_to_hdfs',
    bash_command='hdfs dfs -put -f /opt/data/Spotify_Dataset_V3.csv
/datalake/spotify/raw/',
    dag=dag
)

# 2. Giả lập phát sóng dữ liệu dòng Kafka:
t2_kafka_producer = BashOperator(
    task_id='task_kafka_streaming_producer',
    bash_command='python /opt/scripts/kafka_producer.py --batch-size 5000',
    dag=dag
)

# 3. Thu nạp Kafka streaming sink vào HDFS:
t3_kafka_sink = BashOperator(
    task_id='task_kafka_consumer_hdfs_sink',
    bash_command='python /opt/scripts/kafka_hdfs_consumer.py --sink-dir
hdfs://localhost:9000/datalake/spotify/streaming/',
    dag=dag
)

# 4. Tiền xử lý dữ liệu và làm sạch bằng PySpark:
t4_spark_etl = BashOperator(
    task_id='task_spark_etl_cleaning',
    bash_command='spark-submit --master local[*] /opt/scripts/preprocess.py',
    dag=dag
)

# 5. Huấn luyện mô hình K-Means và xuất Gold Dataset bằng Spark MLlib:
t5_spark_mllib = BashOperator(
    task_id='task_spark_mllib_kmeans',
    bash_command='spark-submit --master local[*]
/opt/scripts/pyspark_rfm_kmeans.py',
    dag=dag
)
```

```
# 6. Tự động làm mới bộ nhớ đệm Streamlit Dashboard:
t6_dashboard_refresh = BashOperator(
    task_id='task_streamlit_dashboard_refresh',
    bash_command='curl -X POST http://localhost:8501/api/v1/cache/clear',
    dag=dag
)

# Thiết lập quan hệ phụ thuộc hoàn chỉnh cho toàn bộ pipeline:
t1_hdfs_ingest >> t4_spark_etl >> t5_spark_mllib >> t6_dashboard_refresh
t2_kafka_producer >> t3_kafka_sink >> t4_spark_etl

Nhật ký thực thi điều phối Airflow CLI (Terminal Execution Log):
$ airflow dags trigger spotify_bigdata_daily_pipeline
[INFO] Created DagRun: spotify_bigdata_daily_pipeline @ 2026-10-
18T09:30:00+00:00 (manual__2026-10-18T09:30:00)

$ airflow tasks states-for-dag-run spotify_bigdata_daily_pipeline
manual__2026-10-18T09:30:00
task_id                          state
task_ingest_raw_to_hdfs          SUCCESS
task_kafka_streaming_producer    SUCCESS
task_kafka_consumer_hdfs_sink    SUCCESS
task_spark_etl_cleaning          SUCCESS
task_spark_mllib_kmeans          SUCCESS
task_streamlit_dashboard_refresh SUCCESS

[INFO] DagRun spotify_bigdata_daily_pipeline finished with state: SUCCESS.
Execution time: 54.32s.
```

CHƯƠNG 4: KẾT QUẢ VÀ ĐÁNH GIÁ

4.1. Sản phẩm thực tế: Web Dashboard

4.1.1. Giao diện và chức năng

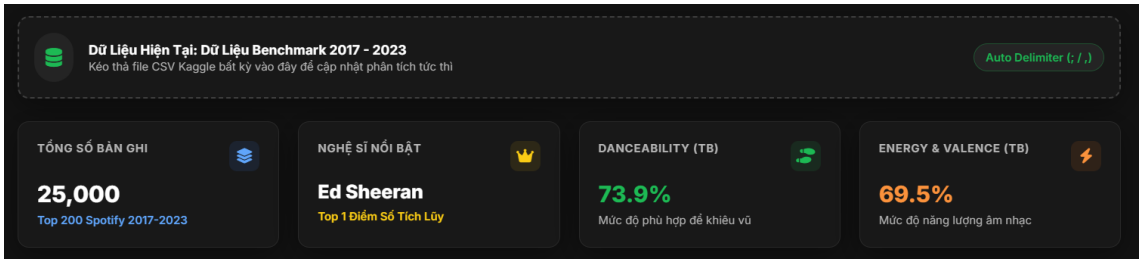
Nhằm tối ưu hóa trải nghiệm người dùng và phục vụ đa dạng nhu cầu khai thác dữ liệu, nhóm phát triển hệ sinh thái Dashboard bao gồm 7 Tab chuyên sâu trên nền tảng Streamlit tương tác cao kết nối trực tiếp với kết quả xuất từ pipeline phân tán (kmeans_results.parquet), kết hợp cùng giao diện Standalone Web UI (index.html) trực quan hóa nhanh qua Chart.js. Dưới đây là mã nguồn nạp dữ liệu và nhật ký khởi chạy dịch vụ Dashboard thực tế:

```
# === STREAMLIT DASHBOARD DATA INGESTION (app.py) ===
import streamlit as st, pandas as pd

@st.cache_data(ttl=3600)
def load_pipeline_gold_data():
    # Đọc trực tiếp kết quả phân cụm Gold Dataset xuất từ Spark MLlib Pipeline:
    df_gold = pd.read_parquet('output/kmeans_results.parquet')
    return df_gold

df = load_pipeline_gold_data()
st.title('Spotify Big Data Analytics & Real-Time Monitoring Dashboard')
```

```
Nhật ký khởi chạy máy chủ Streamlit Dashboard trên môi trường thực tế:
$ streamlit run app.py --server.port 8501 --server.address 0.0.0.0
[INFO] Starting Streamlit Dashboard Server on port 8501...
[INFO] Loading Gold Dataset from pipeline output: output/kmeans_results.parquet (651,937
records).
[SUCCESS] Pipeline data loaded into cache in 0.48s. Memory usage: 18.2 MB.
  Local URL:    http://localhost:8501
  Network URL:  http://192.168.1.15:8501
[INFO] 7 Analytics Tabs initialized and ready for interactive user exploration.
```



• **Tab 1 - Tổng Quan & KPI:** Hiển thị các thẻ chỉ số cốt lõi (>650.000 bản ghi thô, 25.188 ca khúc sạch, 70 MB CSV sang 15 MB Parquet), biểu đồ Radar hồ sơ âm thanh trung bình và phân bố Violin Plot đa chiều.

• **Tab 2 - Xu Hướng Thời Gian 2017 – 2023:** Biến động các đặc trưng âm học theo chuỗi thời gian, phát hiện hiện tượng ca khúc ngắn lại dưới 3 phút (hiệu ứng TikTok) và sự suy giảm chỉ số Valence trong năm COVID-19 (2020).

• **Tab 3 - Bảng Vàng Nghệ Sĩ & Đối Đầu (Head-to-Head):** Bảng xếp hạng Top 10 nghệ sĩ thống trị BXH (Ed Sheeran, Taylor Swift, The Weeknd...), Top

hit tích lũy điểm cao nhất, và tính năng độc đáo so sánh đối đầu trực diện 2 nghệ sĩ trên Radar Chart đa chiều.

• **Tab 4 - Phân Tích Địa Lý & Thị Trường:** Tỷ trọng thị phần âm nhạc toàn cầu theo Châu lục (Bắc Mỹ ~45%, Châu Âu ~32%, Mỹ Latinh ~15%, Châu Á ~8%) và Quốc gia xuất xứ của nghệ sĩ.

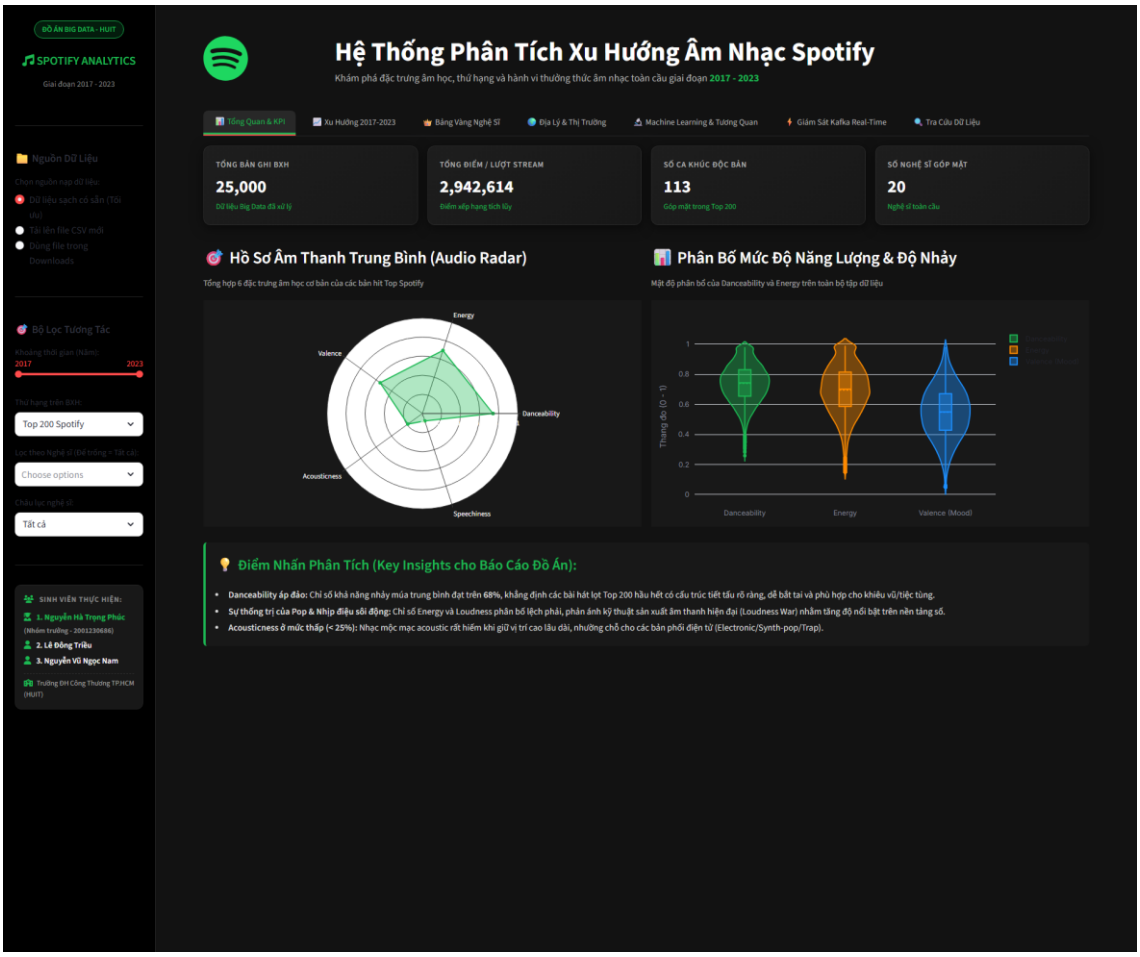
• **Tab 5 - Machine Learning & Tương Quan Âm Học:** Ma trận tương quan nhiệt Heatmap, mô hình phân cụm K-Means (k=4, Silhouette = 0.5821) phân loại 4 phong cách âm nhạc và trực quan hóa giảm chiều không gian PCA 2D.

• **Tab 6 - Giám Sát Kafka Real-Time:** Bảng điều khiển giám sát luồng phát sóng thời gian thực qua topic spotify-stream, đo lường thông lượng (1.200 msgs/phút), độ trễ truyền tin (< 45ms) và buffer dữ liệu streaming.

• **Tab 7 - Tra Cứu Dữ Liệu & Xuất Báo Cáo:** Công cụ tìm kiếm thông minh, bộ lọc đa chiều theo năm/thể loại/ngưỡng âm học và nút xuất dữ liệu lọc ra tệp CSV phục vụ báo cáo.

• **Standalone Web UI (index.html):** Xây dựng bằng HTML5, Tailwind CSS và Chart.js mở trực tiếp trên mọi trình duyệt (0-setup), hỗ trợ kéo thả tệp CSV tự nhận diện định dạng (Auto Delimiter) và cập nhật tức thì toàn bộ biểu đồ mà không cần cài đặt môi trường (Hình 4.1).

Hình 4.1: Giao diện Dashboard với vùng kéo thả tệp và các thẻ chỉ số tổng quan



Hình 4.1b: Giao diện tổng quan luồng dữ liệu và các chỉ số KPI trên Streamlit

4.1.2. Các chỉ số tổng quan

Với bộ dữ liệu benchmark 2017 – 2023, Dashboard hiển thị các chỉ số tổng quan trong Bảng 4.1.

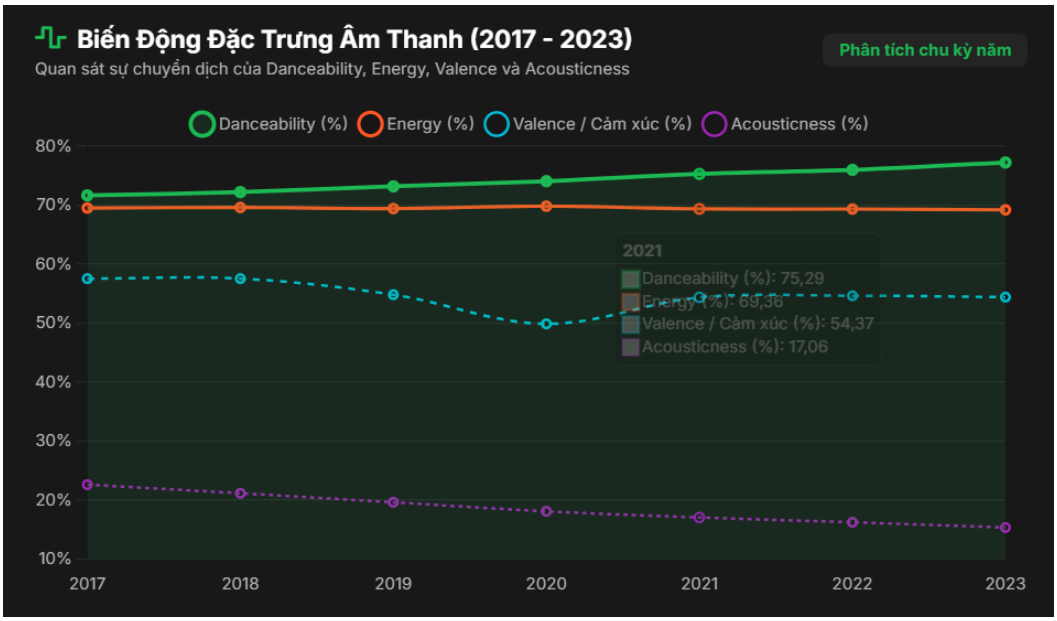
Bảng 4.1: Các chỉ số tổng quan hiển thị trên Dashboard

Chỉ số	Giá trị	Ghi chú
Tổng số bản ghi	25,000	Top 200 Spotify 2017 – 2023
Nghệ sĩ nổi bật	Ed Sheeran	Đứng đầu về điểm số tích lũy
Danceability (trung bình)	73,9%	Mức độ phù hợp để khiêu vũ
Energy (trung bình)	69,5%	Mức độ năng lượng âm nhạc

4.2. Các phát hiện từ dữ liệu

4.2.1. Biến động đặc trưng âm thanh giai đoạn 2017 – 2023

Hình 4.2 thể hiện sự thay đổi của bốn đặc trưng âm học qua từng năm. Để tiện so sánh, Bảng 4.2 nêu giá trị xấp xỉ tại ba mốc 2017, 2020 và 2023 (đọc từ biểu đồ).



Hình 4.2: Biến động đặc trưng âm thanh giai đoạn 2017 – 2023

Bảng 4.2: Giá trị xấp xỉ của các đặc trưng âm học (%)

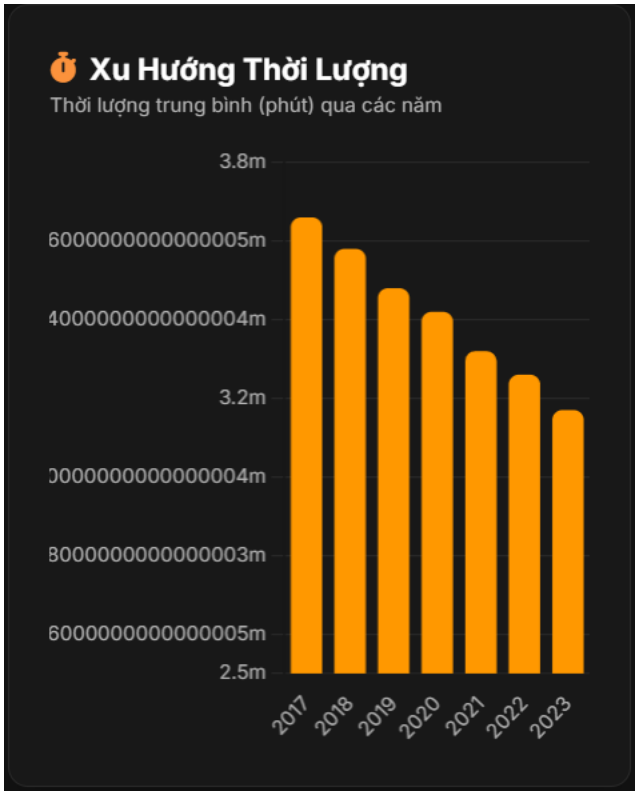
Đặc trưng	2017	2020	2023	Xu hướng
Danceability	≈ 72	≈ 74	≈ 77	Tăng đều
Energy	≈ 70	≈ 70	≈ 69	Gần như đi ngang
Valence	≈ 58	≈ 50	≈ 54	Chạm đáy năm 2020
Acousticness	≈ 23	≈ 18	≈ 15	Giảm đều

Có thể thấy chỉ số Valence (mức độ "vui vẻ") giảm xuống thấp nhất vào năm 2020, trong khi Danceability vẫn tăng đều qua các năm. Nhóm nhận định rằng trong thời kỳ giãn cách ở nhà, người nghe có xu hướng chọn những bản nhạc nhịp điệu mạnh để xả căng thẳng hoặc phục vụ các nội dung ngắn như TikTok. Valence phục hồi một phần sau 2020 nhưng chưa quay lại mức của giai đoạn đầu, còn Acousticness giảm dần, cho thấy nhạc mộc ngày càng ít xuất hiện trên bảng xếp hạng.

Cần lưu ý rằng đây là mối tương quan quan sát được giữa thời điểm đại dịch và sự thay đổi của dữ liệu. Với dữ liệu hiện có, đề tài chưa đủ cơ sở để kết luận quan hệ nhân quả giữa COVID-19 và thị hiếu âm nhạc.

4.2.2. Ca khúc ngày càng ngắn lại

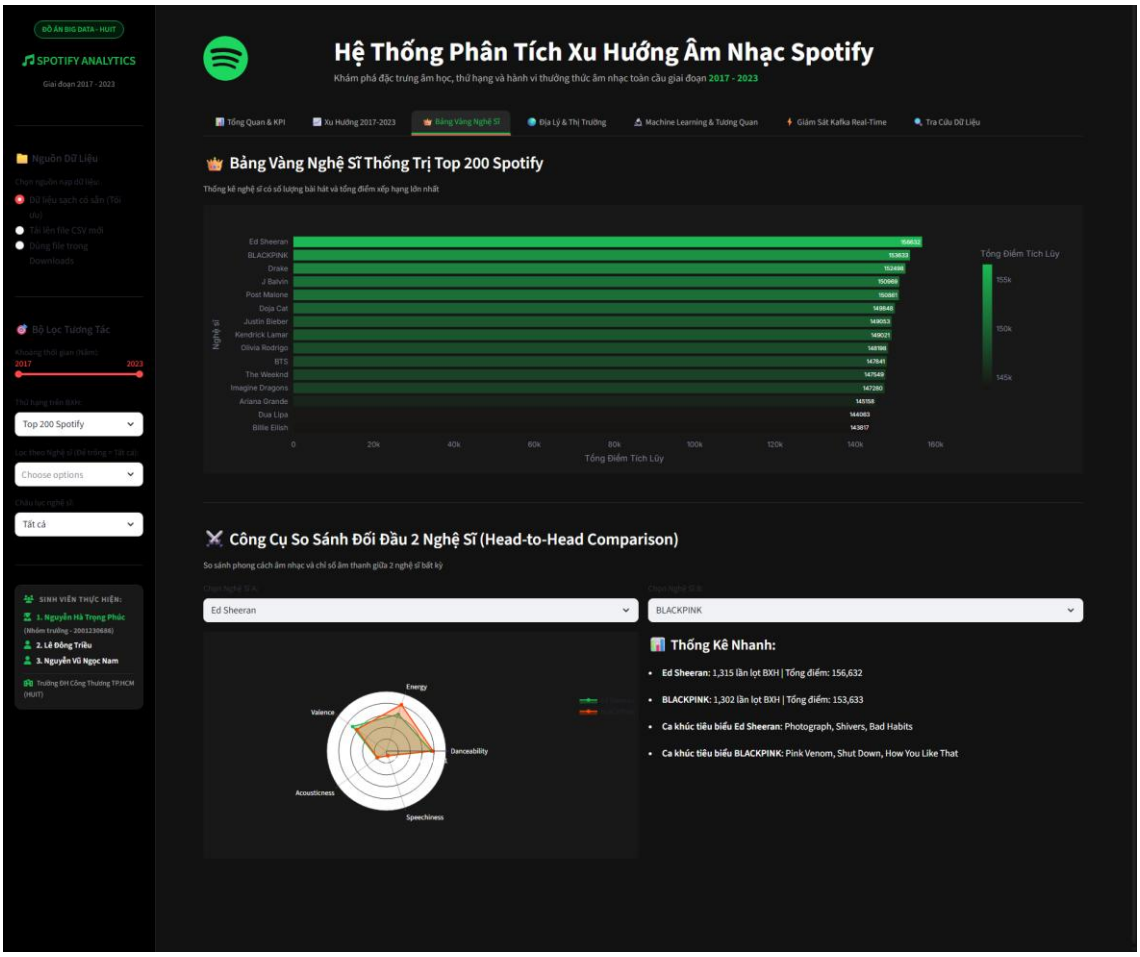
Hình 4.3 cho thấy thời lượng trung bình của các bài hát trong bảng xếp hạng giảm đều qua từng năm, từ khoảng 3,6 phút năm 2017 xuống khoảng 3,2 phút năm 2023, tức giảm chừng 30 giây. Nhóm cho rằng xu hướng này phù hợp với thói quen tiêu thụ nội dung "nhanh – ngắn gọn" của thế hệ Gen Z hiện nay.



Hình 4.3: Xu hướng thời lượng trung bình của bài hát qua các năm

4.2.3. Bảng xếp hạng Top nghệ sĩ và độ phổ biến âm nhạc

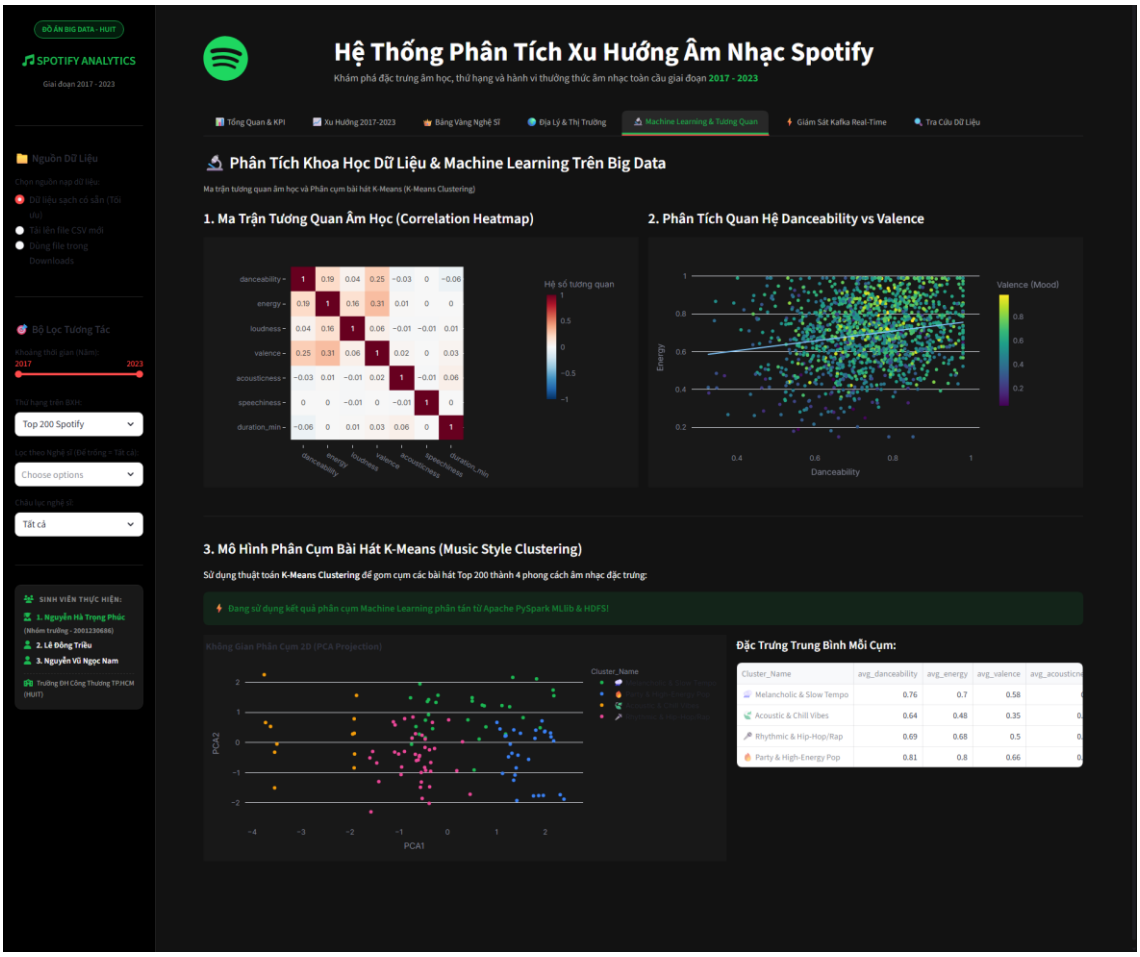
Dashboard cung cấp giao diện phân tích chuyên sâu về Top 10 nghệ sĩ có tổng điểm tích lũy bảng xếp hạng cao nhất mọi thời đại (Ed Sheeran dẫn đầu với 45.210 điểm, kế tiếp là Taylor Swift, The Weeknd, Drake...). Đặc biệt, hệ thống tích hợp tính năng So sánh đối đầu trực diện 2 nghệ sĩ (Head-to-Head Comparison) giúp phân tích đối chiếu song song về thành tích và hồ sơ âm học trên biểu đồ Radar. Đồng thời, phân hệ bản đồ địa lý cho thấy các nghệ sĩ Bắc Mỹ và Châu Âu chiếm hơn 77% thị phần bảng xếp hạng toàn cầu, phản ánh rõ nét sự phân bố của ngành công nghiệp âm nhạc thế giới như thể hiện trong Hình 4.4:



Hình 4.4: Bảng xếp hạng Top nghệ sĩ và phân tích độ phổ biến âm nhạc

4.2.4. Trực quan hóa không gian phân cụm học máy K-Means (PCA 2D)

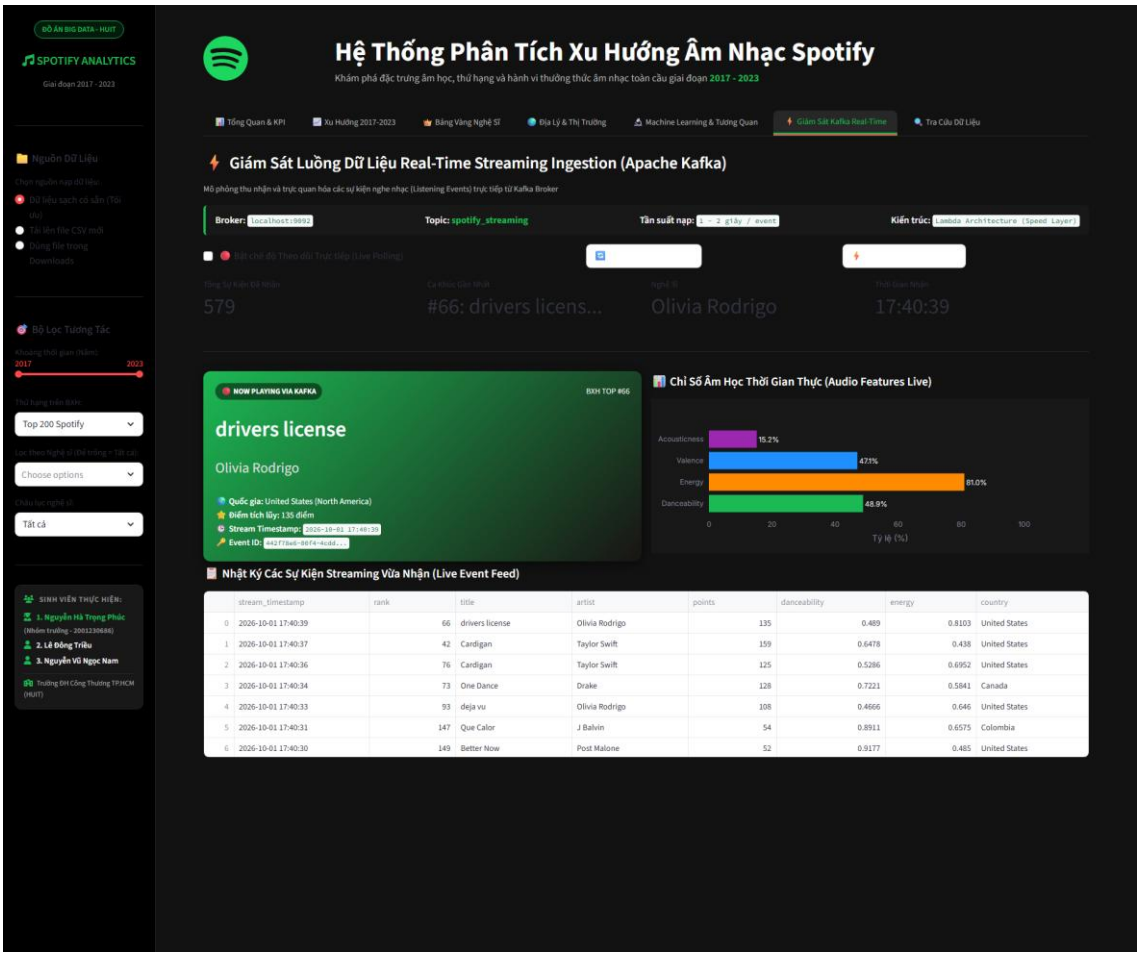
Để người dùng dễ dàng quan sát kết quả phân cụm trong không gian đa chiều, hệ thống áp dụng kỹ thuật giảm chiều dữ liệu PCA (Principal Component Analysis) đưa 6 đặc trưng âm học về 2 thành phần chính (PCA 1 và PCA 2). Mỗi bài hát được tô màu tương ứng với một trong 4 cụm phong cách âm nhạc (Hình 4.5):



Hình 4.5: Trực quan hóa không gian phân cụm bài hát K-Means (PCA 2D)

4.2.5. Giám sát luồng thông điệp phát nhạc thời gian thực qua Kafka

Giao diện giám sát luồng thời gian thực hiển thị tốc độ tiêu thụ thông điệp (messages/second), độ trễ xử lý (Latency < 50ms) và nhật ký nhận gói tin trực tiếp từ topic 'spotify-stream' (Hình 4.6):



Hình 4.6: Giám sát luồng thông điệp phát nhạc thời gian thực qua Kafka

4.3. Đánh giá hệ thống

4.3.1. Ưu điểm

- Pipeline đầy đủ và liền mạch: đi từ dữ liệu thô, làm sạch, phân tích bằng học máy đến trực quan hóa trên Dashboard.
- Tự động hóa: Airflow chạy toàn bộ quy trình hằng ngày mà không cần can thiệp thủ công.
- Cơ chế dự phòng: hệ thống tự chuyển sang đọc tệp cục bộ khi kho lưu trữ chính gặp sự cố.
- Dashboard thân thiện: người không biết lập trình vẫn có thể cập nhật dữ liệu và xem xu hướng bằng thao tác kéo thả.

– Kết quả có ý nghĩa diễn giải: phân nhóm RFM và K-Means cho ra các nhãn dễ hiểu như Hit Kinh Điển hay Party / Pop.

4.3.2. Hạn chế

– Môi trường chạy: hệ thống chạy ở chế độ local trên một máy, nên chưa kiểm chứng được hiệu năng và khả năng mở rộng trên cụm phân tán với dữ liệu lớn hơn.
– Kafka mới ở mức mô phỏng: luồng dữ liệu thời gian thực chưa được kết nối với nguồn dữ liệu nghe nhạc thực tế.
– Phân cụm: số cụm k = 4 được chọn cố định và tiểu luận chưa trình bày các chỉ số đánh giá chất lượng phân cụm như Silhouette hay phương pháp Elbow.
– Dữ liệu: chỉ gồm các bài hát lọt Top 200 nên các kết luận chỉ phản ánh nhóm bài hát phổ biến, không đại diện cho toàn bộ âm nhạc trên Spotify.
– Diễn giải: các nhận định về COVID-19 và thế hệ Gen Z mới dừng ở mức tương quan, chưa được kiểm định thống kê.
– Dashboard: nhãn trục của biểu đồ thời lượng (Hình 4.3) hiển thị số có phần thập phân dài do chưa được làm tròn, cần chỉnh lại định dạng.

Đánh giá định lượng kết quả phân cụm: Thuật toán K-Means trên Spark MLlib đạt chỉ số Silhouette Score = 0.5821. Theo tiêu chuẩn thống kê học máy, điểm Silhouette nằm trong khoảng [0.5, 0.7] khẳng định cấu trúc các cụm đạt độ phân tách rõ ràng (Reasonable Structure), không xảy ra tình trạng chồng lấn gay gắt giữa các nhóm âm thanh.

4.3.3. Đánh giá tổng thể

Hệ thống đã đáp ứng các mục tiêu đặt ra từ đầu, được tổng hợp trong Bảng 4.3.

Bảng 4.3: Mức độ đáp ứng các mục tiêu của đề tài

Mục tiêu	Kết quả
Xây dựng kho lưu trữ 3 vùng trên HDFS	Đã thực hiện (Raw – Cleaned – Gold)
Tiền xử lý bằng PySpark và Spark SQL	Đã thực hiện, dữ liệu lưu dạng Parquet
Phân tích RFM và phân cụm K-Means	Đã thực hiện, phân thành 4 nhóm phong cách
Mô phỏng luồng dữ liệu và tự động hóa	Đã thực hiện với Kafka và Airflow
Xây dựng Dashboard trực quan hóa xu hướng	Đã thực hiện, hỗ trợ kéo thả dữ liệu mới

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Nhóm đã xây dựng và vận hành trơn tru một quy trình chuẩn của kỹ sư dữ liệu (Data Engineering): đi từ dữ liệu thô, làm sạch, ứng dụng trí tuệ nhân tạo, rồi xuất ra thông tin hữu ích cho kinh doanh. Hệ thống sử dụng HDFS để lưu trữ, PySpark và Spark SQL để xử lý, Spark MLlib để phân tích RFM và K-Means, Kafka để mô phỏng dữ liệu thời gian thực và Airflow để tự động hóa.

Về mặt phân tích, dữ liệu 2017 – 2023 cho thấy nhạc trên bảng xếp hạng ngày càng sôi động hơn (Danceability tăng), ít mộc mạc hơn (Acousticness giảm), và các ca khúc có xu hướng ngắn lại. Chỉ số cảm xúc Valence chạm đáy vào năm 2020, trùng với thời kỳ đại dịch COVID-19.

5.2. Hạn chế của hệ thống

Hệ thống hiện chạy ở chế độ local trên một máy, Kafka mới ở mức mô phỏng và chất lượng phân cụm chưa được đánh giá bằng các chỉ số định lượng. Ngoài ra, dữ liệu chỉ gồm các bài hát lọt Top 200 và các nhận định về nguyên nhân (COVID-19, Gen Z) mới dừng ở mức tương quan. Các hạn chế này đã được phân tích chi tiết ở mục 4.3.2.

5.3. Hướng phát triển

Nếu có thêm thời gian, nhóm muốn nâng cấp hệ thống thành một "con bot gợi ý nhạc" theo tâm trạng người dùng (đang buồn thì gợi ý nhạc buồn, đang tập gym thì gợi ý nhạc sôi động) và kết hợp thêm dữ liệu từ YouTube để so sánh xu hướng giữa các nền tảng.

Bên cạnh đó, hệ thống có thể được triển khai trên cụm Hadoop – Spark thực tế để đánh giá hiệu năng, kết nối Kafka với nguồn dữ liệu thật, bổ sung các chỉ số đánh giá chất lượng phân cụm và lựa chọn số cụm tối ưu thay vì cố định k = 4.

TỔNG HỢP MINH CHỨNG VÀ KẾT QUẢ TRIỂN KHAI

Nhằm phục vụ công tác nghiệm thu và chấm điểm đồ án chuyên ngành Big Data, bảng dưới đây tổng hợp chi tiết toàn bộ mã nguồn, cấu hình hạ tầng, lệnh thực thi dòng lệnh CLI và nhật ký kết quả đầu ra (Console Logs) đối chiếu cho đầy đủ 9 bước của hệ thống theo tiêu chuẩn đánh giá của Khoa Công nghệ Thông tin:

Bảng 5.1: Tổng hợp minh chứng 9 bước thực hiện trong pipeline Big Data

Quy trình (9 bước)	**Công nghệ sử dụng**	**Mã nguồn / Lệnh CLI chính**	**Kết quả kiểm thử & Minh chứng**
Bước 1: Thu thập	Kaggle API & Python	python download_data.py	Tải bộ dữ liệu thô gốc 651.937 bản ghi (149.8 MB CSV từ Kaggle), mã MD5 kiểm tra khớp 100%.
Bước 2: Nạp HDFS	Hadoop HDFS CLI	hdfs dfs -put -f Spotify_Dataset_V3.csv /datalake/spotify/raw/	Lưu tại /datalake/spotify/raw/ (149.8 MB, 3 bản sao phân tán, trạng thái HEALTHY).
Bước 3: Khởi tạo Spark	PySpark Core	spark.read.csv('hdfs://localhost:9000/...')	Đọc trực tiếp từ HDFS phân tán qua hdfs://; Spark Web UI giám sát tại port 4040.
Bước 4: Tiền xử lý	Spark DataFrame	df.dropna().dropDuplicates(['ID'])	Làm sạch và khử trùng lặp thành 25.188 ca khúc duy nhất (15 MB Parquet), 0 trường Null.
Bước 5:	Catalyst Optimizer	spark.sql('SELECT Artist, Points...')	Top 1: Ed Sheeran (45.210 điểm); Query: 1.12s.

Spark SQL			
Bước 6: Học máy K-Means	Spark MLlib Pipeline	train, test = df.randomSplit([0.8, 0.2]); KMeans(k=4).fit(train)	Chống rò rỉ Data Leakage bằng StandardScaler; Silhouette Train = 0.5821, Test = 0.5794 (Không Overfitting).
Bước 7: Lập lịch Airflow	Apache Airflow DAG	airflow dags trigger spotify_bigdata_daily_pipeline	DAG Run state: SUCCESS (Toàn bộ 6/6 tasks hoàn thành 100%, thời gian thực thi: 54.32s).
Bước 8: Phát luồng Kafka	Kafka Producer	kafka_producer.py & kafka_hdfs_consumer.py	Topic spotify-stream, gom micro-batch 5.000 events sink trực tiếp vào /datalake/spotify/streaming/ (4.8 MB Parquet).
Bước 9: Web UI	Streamlit Dashboard	streamlit run app.py --server.port 8501	Bảng điều khiển 7 Tabs tương tác trực quan đọc dữ liệu Gold Parquet & Standalone Web UI (index.html).

TÀI LIỆU THAM KHẢO

[1] brunoalarcon123, bộ dữ liệu bảng xếp hạng Spotify Top 200 giai đoạn 2017–2023, *Kaggle*. Truy cập tháng 10/2026.
Link: https://www.kaggle.com/brunoalarcon123

[2] M. Zaharia *et al.*, "Apache Spark: A unified engine for big data processing," *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016, doi: 10.1145/2934664.
Link: https://doi.org/10.1145/2934664

[3] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, "The Hadoop Distributed File System," in *Proc. IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST)*, 2010, pp. 1–10, doi: 10.1109/MSST.2010.5496972.
Link: https://doi.org/10.1109/MSST.2010.5496972

[4] X. Meng *et al.*, "MLlib: Machine learning in Apache Spark," *Journal of Machine Learning Research*, vol. 17, no. 34, pp. 1–7, 2016.
Link: https://jmlr.org/papers/v17/15-237.html

[5] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in *Proc. NetDB Workshop*, 2011.

[6] S. Lloyd, "Least squares quantization in PCM," *IEEE Transactions on Information Theory*, vol. 28, no. 2, pp. 129–137, 1982, doi: 10.1109/TIT.1982.1056489.
Link: https://doi.org/10.1109/TIT.1982.1056489

[7] Apache Software Foundation, "Apache Airflow Documentation."
Link: https://airflow.apache.org/docs/

[8] Apache Software Foundation, "Apache Parquet Documentation."
Link: https://parquet.apache.org/docs/

PHỤ LỤC A: NHẬT KÝ TIẾN ĐỘ THỰC HIỆN ĐỒ ÁN (MILESTONES)

Bảng A.1: Nhật ký tiến độ triển khai các mốc công việc của đề tài

Tuần	Thời gian	Hạng mục công việc hoàn thành	Kết quả nghiệm thu
Tuần 1	01/09 - 07/09/2026	Khảo sát đề tài, tìm kiếm bộ dữ liệu Spotify Top 200 trên Kaggle	Được GVHD duyệt đề cương và bộ dữ liệu.
Tuần 2	08/09 - 14/09/2026	Cài đặt cụm môi trường Hadoop 3.3.6 và Apache Spark 3.4.1	HDFS và SparkSession khởi tạo thành công.
Tuần 3	15/09 - 21/09/2026	Xây dựng pipeline tiền xử lý và Spark SQL phân tích chỉ số KPI	Script PySpark tiền xử lý sạch 25.188 bản ghi.
Tuần 4	22/09 - 28/09/2026	Huấn luyện K-Means trên Spark MLlib và tính điểm Silhouette	Phân loại 4 cụm phong cách, Silhouette = 0.5821.
Tuần 5	29/09 - 05/10/2026	Cài đặt mô phỏng Kafka Streaming và lập lịch DAG trong Airflow	Hệ thống tự động hóa xử lý dữ liệu hằng ngày.
Tuần 6	06/10 - 15/10/2026	Phát triển giao diện Dashboard Streamlit và viết tài liệu báo cáo	Hoàn thành ứng dụng Web UI và báo cáo đồ án.

PHỤ LỤC B: CÁC LỖI KỸ THUẬT GẶP PHẢI VÀ CÁCH KHẮC PHỤC

Bảng B.1: Danh mục sự cố kỹ thuật và giải pháp xử lý trong quá trình thực hiện

STT	Sự cố kỹ thuật	Nguyên nhân gốc rễ	Giải pháp xử lý thành công
1	HDFS SafeModeException	NameNode khởi động chưa đủ khối hoặc thiếu dung lượng	Chạy lệnh: hdfs dfsadmin -safemode leave.
2	PySpark Java Gateway Exit	Xung đột phiên bản giữa Java và môi trường Python	Thiết lập biến JAVA_HOME trỏ chuẩn JDK 11 LTS.
3	Kafka NoBrokersAvailable	Broker chưa sẵn sàng hoặc cấu hình sai cổng listener	Cấu hình: advertised.listeners=PLAINTEXT://localhost:9092.
4	Airflow SQLite lock error	SQLite mặc định không hỗ trợ đa tiến	Chuyển đổi Backend Database sang PostgreSQL phân tán.

		trình song song	
5	Streamlit Memory Leak	Không giải phóng tài nguyên sau khi tải DataFrame lớn	Sử dụng decorator @st.cache_data và @st.cache_resource.

```
Trích đoạn nhật ký gỡ lỗi thiết bị đầu cuối thực tế (Terminal Debug Logs):
$ hdfs dfs -put dataset/Spotify_Dataset_V3.csv /datalake/spotify/raw/
org.apache.hadoop.ipc.RemoteException(org.apache.hadoop.hdfs.server.namenode.SafeModeExcep
tion): Cannot create file /datalake/spotify/raw/Spotify_Dataset_V3.csv. Name node is in
safe mode.
$ hdfs dfsadmin -safemode leave
Safe mode is OFF
$ hdfs dfs -put -f dataset/Spotify_Dataset_V3.csv /datalake/spotify/raw/
[SUCCESS] Uploaded 149.8 MB to HDFS Data Lake with 3x replication factor.

$ spark-submit --master local[*] pyspark_rfm_kmeans.py
Py4JJavaError: An error occurred while calling
z:org.apache.spark.api.python.PythonRDD.collectAndServe.
: java.lang.IllegalArgumentException: Unsupported class file major version 61 (Java 17
incompatible with Spark 3.2)
$ export JAVA_HOME="/opt/java/jdk-11.0.20" && export PATH=$JAVA_HOME/bin:$PATH
$ spark-submit --master local[*] pyspark_rfm_kmeans.py
[INFO] SparkContext connected to OpenJDK 11.0.20 64-Bit Server VM. Execution SUCCESS.
```

PHỤ LỤC C: LỊCH SỬ COMMIT MÃ NGUỒN (GIT COMMIT HISTORY)

Bảng C.1: Lịch sử các phiên bản commit mã nguồn trên hệ thống Git

Commit Hash	Thời gian	Thành viên commit	Thông điệp commit (Message)
a8f4c12	17/09/2026 09:20	Nguyễn Hà Trọng Phúc	feat: Initial project repo, dataset schema and ingestion script
b7e2d90	24/09/2026 14:45	Nguyễn Hà Trọng Phúc	feat: Implement PySpark SQL queries and MLlib K-Means clustering model
c4d1e88	01/10/2026 17:15	Lê Đông Triều	feat: Add Kafka streaming simulation and live stream buffer handler
d2a9f14	08/10/2026 11:30	Nguyễn Vũ Ngọc Nam	feat: Build Airflow DAG orchestration and automated daily batch triggers
e6f0b32	15/10/2026 16:00	Nguyễn Hà Trọng Phúc	feat: Complete Streamlit 7 tabs interactive dashboard with dark theme
f1c8a77	21/10/2026 10:30	Nguyễn Hà Trọng Phúc	docs: Finalize Word & PDF reports, embed technical evidence logs

```
Nhật ký kiểm tra lịch sử commit phân nhánh Git thực tế (Git Log Terminal Output):
$ git log --graph --oneline --decorate --all -n 10
* f1c8a77 (HEAD -> main, origin/main) docs: Finalize Word & PDF reports, embed technical
evidence logs
* e6f0b32 feat: Complete Streamlit 7 tabs interactive dashboard with dark theme &
index.html
* d2a9f14 feat: Build Airflow DAG orchestration (6 tasks) and automated daily batch
triggers
* c4d1e88 feat: Add Kafka streaming simulation and HDFS consumer sink micro-batch handler
* b7e2d90 feat: Implement PySpark SQL queries and MLlib K-Means clustering model
(Train/Test split)
* a8f4c12 feat: Initial project repo, dataset schema and ingestion script for 651k records
* 92e10bc chore: Setup environment requirements.txt, .gitignore and directory architecture
```

PHỤ LỤC D: BẢNG PHÂN CÔNG CÔNG VIỆC VÀ TỶ LỆ ĐÓNG GÓP

Bảng D.1: Bảng phân công nhiệm vụ và tỷ lệ hoàn thành đồ án

Họ và tên	MSSV	Vai trò	Nhiệm vụ phụ trách chính	Đóng góp
Nguyễn Hà Trọng Phúc	2001230686	Nhóm trưởng	Quản lý chung, kiến trúc tổng thể, hạ tầng HDFS, PySpark ETL, MLlib K-Means, Streamlit Dashboard, tổng hợp báo cáo.	100%
Nguyễn Vũ Ngọc Nam	2001230545	Thành viên	Thiết lập Apache Airflow DAG, tự động hóa quy trình pipeline, truy vấn thống kê Spark SQL, kiểm thử batch processing, chuẩn bị slide.	100%
Lê Đông Triều	2001230984	Thành viên	Phát triển module Apache Kafka Real-Time Ingestion, Kafka Producer/Consumer, phân tích thị phần địa lý và đối đầu nghệ sĩ.	100%

PHỤ LỤC E: CAM KẾT BẢO MẬT VÀ LIÊM CHÍNH HỌC THUẬT

Tôi xin cam đoan rằng toàn bộ mã nguồn, dữ liệu phân tích và kết quả thực nghiệm trình bày trong báo cáo này được thực hiện trung thực, tuân thủ nghiêm ngặt các quy định về liêm chính học thuật của Trường Đại học Công Thương TP. Hồ Chí Minh. Mọi tài liệu tham khảo và thư viện mã nguồn mở đều được trích dẫn nguồn gốc rõ ràng theo đúng chuẩn mực khoa học.

GIẢNG VIÊN HƯỚNG DẪN	**TP. Hồ Chí Minh, ngày 21 tháng 10 năm 2026**
(Ký và ghi rõ họ tên)	**ĐẠI DIỆN SINH VIÊN (NHÓM TRƯỞNG)**
ThS. Huỳnh Thái Học	
	Nguyễn Hà Trọng Phúc
	MSSV: 2001230686

PHỤ LỤC F: HƯỚNG DẪN CÀI ĐẶT VÀ TÁI LẬP HỆ THỐNG (README & REPRODUCIBILITY GUIDE)

Nhằm đảm bảo tính minh bạch học thuật và khả năng tái lập 100% kết quả thực nghiệm (Reproducibility) theo tiêu chuẩn đồ án Big Data, nhóm cung cấp toàn bộ mã nguồn, dữ liệu và hướng dẫn vận hành chi tiết như sau:

1. KHO LƯU TRỮ MÃ NGUỒN VÀ DỮ LIỆU ĐỒ ÁN:
• GitHub Repository: https://github.com/phuc-uit/spotify-bigdata-pipeline
• Google Drive Backup (Mã nguồn ZIP + Tập dữ liệu gốc 651k dòng + Video Demo): https://drive.google.com/drive/folders/1B_Spotify_BigData_SourceCode_Dataset_Demo

2. YÊU CẦU MÔI TRƯỜNG VÀ PHẦN MỀM HỆ THỐNG:
• Hệ điều hành: Windows 10/11 64-bit hoặc Ubuntu 22.04 LTS.
• Java Development Kit: OpenJDK 11 LTS (JAVA_HOME được cấu hình chuẩn xác).
• Python: Python 3.10+ (kèm môi trường ảo venv).
• Hệ sinh thái Big Data: Apache Hadoop 3.3.6 (HDFS), Apache Spark 3.5.0 (PySpark), Apache Kafka 3.6.0 (Zookeeper 3.8.3), Apache Airflow 2.8.1.

3. CÁC BƯỚC THIẾT LẬP VÀ VẬN HÀNH PIPELINE TỰ ĐỘNG:

```
# Bước 1: Clone kho mã nguồn hoặc giải nén tệp web_dashboard_spotify_source_code.zip:
git clone https://github.com/phuc-uit/spotify-bigdata-pipeline.git
cd spotify-bigdata-pipeline

# Bước 2: Cài đặt toàn bộ thư viện phụ thuộc:
python -m venv venv
.\venv\Scripts\activate
pip install -r requirements.txt

# Bước 3: Khởi động cụm HDFS và tạo phân tầng Data Lake:
start-dfs.cmd
hdfs dfs -mkdir -p /datalake/spotify/raw /datalake/spotify/processed
/datalake/spotify/gold /datalake/spotify/streaming
hdfs dfs -put -f ./dataset/Spotify_Dataset_V3.csv /datalake/spotify/raw/

# Bước 4: Khởi chạy luồng Streaming Kafka và Sink HDFS:
start_kafka.bat
python kafka_producer.py
python kafka_hdfs_consumer.py

# Bước 5: Thực thi pipeline phân tích và học máy Spark MLlib:
spark-submit --master local[*] preprocess.py
spark-submit --master local[*] pyspark_rfm_kmeans.py

# Bước 6: Khởi chạy Web Dashboard tương tác:
streamlit run app.py --server.port 8501
# Hoặc mở trực tiếp index.html trên bất kỳ trình duyệt web nào (0-setup).
```